

PRÁCTICA 2

CONEXIA: Plataforma de conexión y optimización de redes.

Programación Orientada a Objetos

**Grupo 01
Equipo 10**

**Maria José Monroy Mejía
Valeria Moreno Rojas
Justin Camilo Loaiza Lujan**

Jaime Alberto Guzmán Luna

Universidad Nacional de Colombia

Sede Medellín

2025

Índice

1. Portada	1
2. Descripción general de la solución	4
2.1. Identificación y requisitos funcionales del problema	4
2.2. Enfoque del sistema	4
2.3. Diseño del sistema	4
3. Descripción del diseño estático del sistema en la especificación UML	6
3.1. Cliente	6
3.2. ProveedorInternet	7
3.3. Plano	8
3.4. Plan	8
3.5. Factura	9
3.6. Servidor	9
3.7. Router	10
3.8. Dispositivo	11
3.9. Antena	11
3.10. Cobertura (Abstracta)	12
3.11. Red (Interfaz)	12
4. Descripción de la implementación de características de POO	14
4.1. Clase abstracta y método abstracto	14
4.2. Herencia múltiple (“Interfaz”)	14
4.3. Herencia	15
4.4. Ligadura dinámica (2 casos)	16
4.5. Serialización y deserialización	17
4.6. Atributos de clase y métodos de clase	18
4.7. Uso de constante	18
4.8. Encapsulamiento	19
4.9. Sobrecarga (1 de métodos y 2 de constructores)	20
4.10. Referencias self (para desambiguar)	22
4.11. Implementación de un caso de enumeración	22
5. Descripción de cada una de las 5 funcionalidades implementadas	24
5.1. Funcionalidad 1: Adquisición de un plan	25
5.2. Funcionalidad 2: Visualizar los dispositivos conectados	29
5.3. Funcionalidad 3: Mejora tu plan	32
5.4. Funcionalidad 4: Test	34
5.5. Funcionalidad 5: Reporte de fallas en el servidor	40
6. Manejo de excepciones	44
6.1. ErrorAplicacion	44
6.2. Exception1 (hereda de ErrorAplicacion)	44
6.3. Exception2 (hereda de ErrorAplicacion)	46
7. FieldFrame	48
7.1. Descripción	48
7.2. Ejemplos de uso en la Interfaz	49

8. Manual de usuario	51
8.1. “Adquisición de un Plan”	52
8.2. “Visualizar dispositivos Vinculados”	55
8.3. “Mejora tu plan”	56
8.4. “Test de velocidad”	58
8.5. “Reporte de fallas en el servidor”	61

2. Descripción general de la solución.

2.1. Identificación y requisitos funcionales del problema.

El problema que se busca solucionar es la falta de rapidez en la respuesta al usuario, ya que procesos como cambiar de plan, adquirir un router o reportar problemas con el servicio pueden ser muy demorados debido a la gestión administrativa. Esta demora afecta la experiencia del usuario y la eficiencia operativa de la empresa. Para abordar esta problemática, se está desarrollando una aplicación que permite a los usuarios realizar múltiples acciones en un solo lugar, ahorrando tiempo y mejorando su experiencia con nuestra empresa. De esta manera, la aplicación facilita la modificación de servicios, la adquisición de planes, la mejora de la conectividad y la resolución de incidencias de forma más ágil y eficiente.

2.2. Enfoque del sistema.

El dominio del proyecto está centrado en una empresa llamada Conexia, dedicada a la distribución y modificación de planes de Internet. La solución tiene como objetivo principal facilitar la modificación y adquisición de planes de Internet, mediante la implementación de cinco funcionalidades principales:

1. Adquisición de un plan: Permite a los usuarios seleccionar un proveedor y un plan de servicio (Basic, Standard o Premium).
2. Visualización de dispositivos conectados: Muestra los dispositivos que están conectados a la red del usuario.
3. Mejora de un plan: Ofrece recomendaciones para mejorar el plan de acuerdo con el uso actual del servicio.
4. Prueba de velocidad de Internet (test): Realiza una prueba de velocidad de conexión y proporciona recomendaciones.
5. Informe o reporte de anomalías: Genera un informe sobre posibles problemas en los servidores de los proveedores.

El sistema está diseñado para ejecutarse en un entorno de usuario único, lo que significa que todas las funcionalidades están dirigidas a mejorar la experiencia de un solo usuario, que interactúa con la consola desde diferentes maneras y propósitos.

2.3. Diseño del sistema.

El diseño del sistema de redes está modelado en un plano cartesiano de 100x100, que representa una abstracción de la realidad. Este espacio incluye cuatro localidades principales, denominadas A, B, C y D, donde se distribuyen componentes clave como antenas, servidores y routers. Estos elementos interactúan de manera lógica para soportar las funcionalidades de la aplicación.

Las antenas tienen zonas de cobertura representadas como circunferencias, cuyo radio varía según la generación de tecnología utilizada:

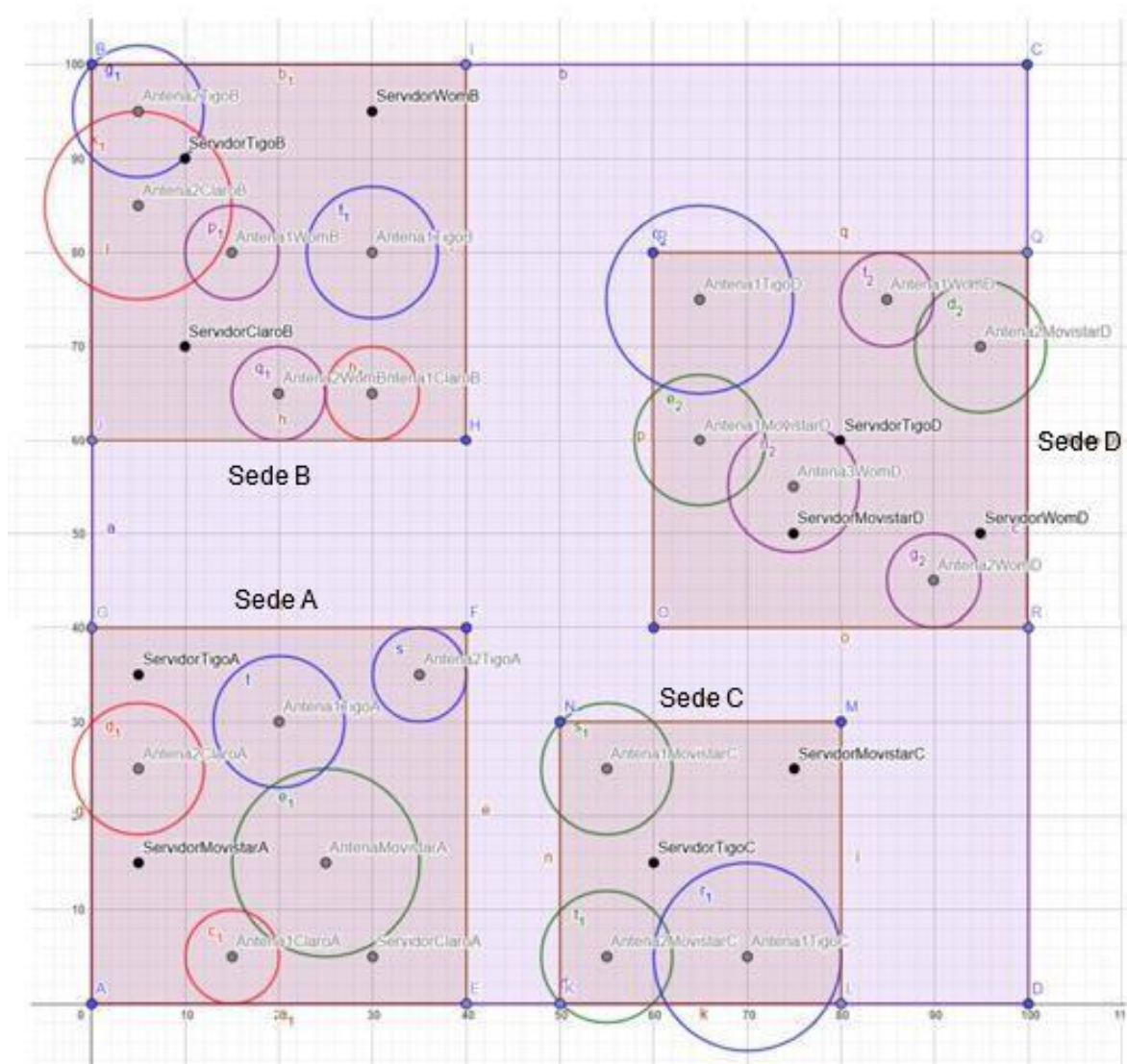
- 3G: radio de 5 unidades.
- 4G: radio de 7 unidades.
- 5G: radio de 10 unidades.

Además, los proveedores de servicio están identificados mediante colores específicos en el esquema:

- Azul para Tigo.
- Verde para Movistar.
- Rojo para Claro.
- Morado para Wom.

Este diseño permite a los usuarios realizar mejoras en su servicio, visualizar los dispositivos conectados a su red y gestionar sus planes de manera más eficiente y rápida, optimizando los tiempos y mejorando significativamente la experiencia general.

A continuación, se presenta un esquema que ilustra la distribución de las antenas, routers y zonas de cobertura dentro del sistema. Este gráfico muestra cómo se organizan los elementos clave en el plano cartesiano.



En caso de necesitar mayor detalle, se puede consultar el siguiente enlace:

<https://www.geogebra.org/classic/kmd7rzk5>

para el cliente.

- Los Getters y Setters: Son para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Router: Representa el modem asociado al cliente.
- ProveedorInternet: Define el proveedor del servicio de internet.
- Factura: Registra los costos y detalles del plan contratado.
- Antena y Servidor: Son utilizados en la configuración de los planes y enrutamiento de la conexión.

3.2. ProveedorInternet.

La clase ProveedorInternet los proveedores de servicios de internet disponibles en el sistema. Esta clase se encarga de manejar información relacionada con los planes de servicio ofrecidos, los clientes asociados y la capacidad de administración de recursos. Además, incluye funcionalidades para verificar clientes, consultar planes disponibles y gestionar la asignación de clientes.

Métodos Principales:

- verificarCliente (Instancia): Verifica si un cliente con un ID específico está asociado al proveedor y tiene un plan activo (BASIC, STANDARD o PREMIUM). También permite comprobar clientes por nombre e ID, validando que tengan más de dos dispositivos conectados.
- planesDisponibles (Instancia): Determina qué planes están disponibles para un cliente, basándose en los cupos restantes en cada plan.
- crearCliente (Instancia): Permite crear un nuevo cliente asociado al proveedor con el nombre e ID especificados.
- proveedorSede (Estático): Encuentra todos los proveedores que operan en una sede específica, utilizando los servidores registrados.
- proveedores (Estático): Actualiza la lista de proveedores totales extrayendo los proveedores registrados en los servidores.
- Los Getters y Setters: Métodos para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Con Cliente: Paraverificar clientes (verificarCliente), crear nuevos clientes (crearCliente) y consultar qué clientes están vinculados a un plan específico.
- Con Plan: Cada proveedor gestiona los planes de servicio (BASIC, STANDARD, PREMIUM) que ofrece a los clientes.
- Con Servidor: Los servidores están asociados a un ProveedorInternet y distribuyen los recursos de red a los clientes del proveedor.
- Con Dispositivo: los dispositivos conectados a un cliente están vinculados al proveedor a través del router y los planes asociados.
- Con Router: Los routers asociados a los clientes y estos a proveedor y los planes contratados.
- Con Antena: Las antenas asociadas al proveedor aseguran la cobertura de red en áreas específicas.
- Con Factura: Las facturas generadas para los clientes del proveedor dependen de los planes contratados y los recursos utilizados.

3.3. Plano.

La clase Plano define las coordenadas y zonas de cobertura geográfica asociadas al sistema. Esta clase es utilizada para establecer la ubicación física de clientes, servidores, antenas y routers dentro del sistema.

Métodos Principales:

- crearSede (Estático): Permite crear un rectángulo que representa la forma y los límites de una sede.
- crearZonaCobertura (Instancia): Genera una zona de cobertura circular que es utilizada por elementos como antenas.
- crearPunto (Instancia): Convierte coordenadas en un objeto Point2D para facilitar operaciones geométricas y cálculos.
- Los Getters y Setters: Métodos para acceder y modificar los atributos privados de la clase.

Relación con Otras Clases:

- Con Cliente: Los clientes se ubican dentro del plano y tienen coordenadas asignadas para determinar su ubicación.
- Con Servidor: Los servidores también están ubicados en el plano, y su posición se determina en función de la sede y la cobertura de red.
- Con Antena: Las antenas tienen una zona de cobertura definida en el plano.
- Con Router: Los routers están ubicados dentro de la zona de cobertura definida en el plano, para proporcionar acceso a los clientes.
- Con ProveedorInternet: Los proveedores gestionan los límites y zonas de cobertura, las cuales se visualizan en el plano mediante las sedes y zonas de cobertura.

3.4. Plan.

La clase Plan describe los tipos de planes de servicio que los proveedores de internet ofrecen a los clientes. Los planes se dividen en tres categorías: BASIC, STANDARD y PREMIUM. Cada plan contiene información sobre la velocidad de subida, velocidad de bajada y el precio del servicio.

Métodos Principales:

- Getters: Que retornan listas con los detalles del plan, incluyendo velocidad de subida, velocidad de bajada y precio.

Relación con Otras Clases:

- Con ProveedorInternet: Los planes de servicio (BASIC, STANDARD, PREMIUM) son gestionados y ofrecidos a través de los proveedores.
- Con Cliente: Los clientes contratan planes, dependiendo de sus necesidades.
- Con Factura: La factura del cliente se genera según el plan contratado.
- Con Router: Los routers están asociados a los planes de servicio para proporcionar acceso a internet.
- Con Dispositivo: Los dispositivos de los clientes están vinculados al plan contratado para acceso a los servicios.

3.5. Factura.

La clase Factura registra los datos financieros y administrativos de los clientes. Esta clase es responsable de manejar información relacionada con los pagos atrasados, el precio total del servicio y la fecha de activación del cliente. Además, incluye métodos para verificar la validez de una factura, gestionar pagos y generar nuevas facturas.

Métodos Principales:

- verificarFactura (Instancia): Verifica si una factura tiene menos de tres pagos atrasados para determinar si es válida.
- accionesPagos (Instancia): Maneja pagos de una factura, ajusta el precio y los pagos atrasados. Devuelve un módem si el pago es válido. Si el abono es incorrecto o insuficiente, no realiza cambios o lanza un error.
- generarFactura (Instancia): Genera una nueva factura basada en el plan de servicio contratado y el proveedor asociado al cliente.
- toString (Instancia): Devuelve una representación textual detallada de la factura, incluyendo el nombre del cliente, IP, pagos atrasados y precio total.
- Los Getters y Setters: Para acceder y modificar los atributos privados de la clase, como el usuario, la IP, los pagos atrasados, el precio y el mes de activación.

Relación con Otras Clases:

- Con Cliente: Cada factura está asociada a un cliente, quien es el responsable del pago del servicio que se le ha proporcionado según el plan contratado.
- Con ProveedorInternet: El proveedor emite las facturas a los clientes por los servicios que proporciona, considerando los planes y los recursos utilizados.
- Con Plan: El costo de la factura está determinado por el plan contratado por el cliente, ya sea BASIC, STANDARD o PREMIUM.
- Con Dispositivo: Los dispositivos asociados a un cliente pueden influir en el cálculo de la factura, especialmente si afectan el consumo de recursos o servicios adicionales.
- Con Router: El uso del servicio a través de los routers puede tener un impacto en el monto de la factura, dependiendo de las condiciones del contrato o el consumo.

3.6. Servidor.

La clase Servidor gestiona la infraestructura necesaria para las conexiones de red y la asignación de recursos en el sistema. Cada servidor está asociado a múltiples routers y tien

Métodos Principales:

- buscarServidores (Estático): Busca y retorna todos los servidores disponibles en una sede específica que no estén saturados.
- verificarAdmin (Instancia): Verifica si un proveedor tiene servidores registrados en el sistema y retorna estos servidores en todas las localidades.
- distanciasOptimas (Instancia): Calcula las distancias óptimas en las que un servidor debe ubicarse para maximizar la intensidad del flujo de red.
- adminServidor (Estático): Instancia un objeto servidor aleatorio que puede usarse para invocar métodos como verificarAdmin.
- Los Getters y Setters: Permiten acceder y modificar los atributos privados de la clase, como la sede, el porcentaje de eficiencia, el estado de saturación, el proveedor asociado, las coordenadas y la lista de routers.

Relación con Otras Clases:

- Con ProveedorInternet: Cada servidor está vinculado a un proveedor de internet y administra las conexiones de sus clientes.
- Con Router: Un servidor posee múltiples routers que se conectan directamente a los clientes.
- Con Plano: Define la ubicación geográfica del servidor para calcular distancias y zonas de cobertura.
- La clase Servidor asegura que los recursos de red se utilicen de manera eficiente, optimizando la calidad del servicio para los clientes mientras gestiona la capacidad máxima del sistema.

3.7. Router.

La clase Router maneja la conectividad de red entre los clientes y los servidores. y está asociado a un servidor y a una antena.

Métodos Principales:

- actualizarVelocidad (Instancia): Ajusta las velocidades de subida y bajada del cliente en función de los dispositivos conectados y si el cliente se encuentra dentro de la zona de cobertura de la antena asociada.
- actualizarPing (Instancia): Calcula y actualiza el valor del ping del router dependiendo de las velocidades actuales.
- test (Instancia): Realiza un diagnóstico general del router llamando a métodos como actualizarVelocidad y actualizarPing. También evalúa condiciones como saturación, cobertura y compatibilidad de generación.
- verificarDispositivos (Instancia): Identifica los dispositivos conectados al router en función de su IP.
- protocoloSeleccionServidor (Instancia): Evalúa las condiciones actuales del router y su servidor asociado para recomendar un servidor diferente si hay otro mejor o más cercano.
- intensidadFlujoOptima (Instancia): Calcula la intensidad del flujo de datos en el servidor más cercano, considerando la distancia, la velocidad de conexión y la eficiencia del servidor, y devuelve el servidor y la intensidad calculada.
- intensidadFlujoClientes (Instancia): Calcula y devuelve una lista de intensidades del flujo de datos para cada cliente, considerando la velocidad de conexión, la distancia al servidor, la eficiencia del servidor y el número de dispositivos conectados. Si reales es True, devuelve la intensidad calculada; si no, devuelve un valor ajustado.
- rastrearGeneracionCompatible (Instancia): Su nombre sugiere rastrear antenas compatibles con una generación específica, pero simplemente retorna None.
- adminRouter (Estático): Crea un router aleatorio que podrá ser utilizado por el administrador para pruebas y reportes
- Los Getters y Setters: Proporcionan acceso a los atributos privados de la clase, como la dirección IP, velocidades, estado en línea, coordenadas, servidor asociado, antena asociada y velocidad total.

Relación con Otras Clases:

- Con Servidor: Un router está directamente vinculado a un servidor, que le proporciona los recursos de red.
- Con Antena: Está asociado a una antena que define su zona de cobertura.
- Con Cliente: Permite la conectividad de los dispositivos del cliente al servidor y gestiona las velocidades asignadas según los planes contratados.

3.8. Dispositivo.

La clase Dispositivo representa los dispositivos conectados a la red. Cada dispositivo tiene información como su dirección IP, nombre, generación tecnológica, y está conectado a un router que gestiona su acceso a la red. Esta clase desempeña un papel importante en funcionalidades como visualizar dispositivos, realizar pruebas de red y optimizar planes de servicio.

Métodos Principales:

- desconectarDispositivo (Estático): Permite desconectar un dispositivo específico de la red asociada al cliente.
- toString (Instancia): Proporciona una representación textual del dispositivo, incluyendo su nombre, dirección IP y generación tecnológica.
- Los Getters y Setters: Proveen acceso a los atributos privados de la clase, como el router asociado, dirección IP, nombre, generación y la lista estática de dispositivos totales.

Relación con Otras Clases:

- Con Router: Cada dispositivo está conectado a un router que gestiona su conexión a la red.
- Con Cliente: Los dispositivos son propiedad de los clientes y están vinculados al router del cliente para acceder al servicio.

3.9. Antena.

La clase Antena gestiona la transmisión de señales dentro de una zona de cobertura específica. Cada antena está asociada a un proveedor de internet y se conecta con routers que distribuyen la señal.

Métodos Principales:

- antenasSede (Estático): Busca las antenas en una sede específica. Si se indica un proveedor, devuelve la primera antena que le pertenezca.
- rastrearGeneracionCompatible (Instancia, sobrescrito): Identifica una antena dentro de una lista que sea compatible con la generación del router del cliente y cuya zona de cobertura incluya al cliente.
- verificarZonaCobertura (Instancia): Verifica si un router se encuentra dentro de la zona de cobertura de la antena.
- intensidadFlujoClientes (Abstracto, heredado de Cobertura): No realiza ninguna acción en la clase Antena, y debe ser implementado por las subclases de Antena.
- intensidadFlujoOptima (Abstracto, heredado de Cobertura): Debe implementarse en las subclases de Antena para calcular la intensidad del flujo óptimo entre los clientes y servidores.
- toString (Instancia, sobrescrito de Cobertura): Proporciona una descripción textual de la antena, incluyendo su identificador, sede, ubicación y generación tecnológica.
- antenas (Estático): Resetea las referencias de los proveedores de las antenas al deserializar, para evitar problemas relacionados con referencias obsoletas.
- Los Getters y Setters: Proveen acceso a los atributos privados de la clase, como el identificador, coordenadas, sede, zona de cobertura y proveedor.

Relación con Otras Clases:

- Con ProveedorInternet: Cada antena está asociada a un proveedor, que define las características del servicio que ofrece.
- Con Router: Las antenas conectan los routers que distribuyen la señal a los clientes.
- Con Plano: Define la ubicación geográfica de la antena y su zona de cobertura.

3.10. Cobertura (Abstracta).

La clase Cobertura es abstracta, define las características generales relacionadas con la intensidad y la generación de flujo de red. Es una clase base que no puede ser instanciada directamente, pero proporciona una estructura común y métodos abstractos que son implementados por sus subclases, como Antena y Router.

Métodos Principales:

- rastrearGeneracionCompatible (Abstracto): es utilizado por la clase Antena en la funcionalidad de pruebas (test). Este método busca una antena compatible con la generación del router del cliente.
- intensidadFlujoOptima (Abstracto): Implementado por Router para la funcionalidad de optimización del plan de servicio. Calcula la intensidad de flujo óptima en función de los servidores y las características del cliente.
- intensidadFlujoClientes (Abstracto): Implementado por Router en la funcionalidad de reportes. Calcula la intensidad de flujo que los clientes deberían recibir en comparación con la intensidad real.
- toString (Sobrescrito): Devuelve una representación textual de la clase. Se puede personalizar en las subclases para mostrar información específica.
- Los Getters y Setters: Proporcionan acceso a los atributos privados de la clase.

Relación con Otras Clases:

- Subclases: Antena y Router usan esta clase para implementar comportamientos específicos, en cada una.
- Con Cliente y Servidor: Colabora con estas clases para calcular la intensidad de flujo, rastrear antenas.
- Con Antena: Define los métodos necesarios para rastrear generaciones compatibles dentro de la cobertura.

3.11. Red (Interfaz).

La interfaz Red declara métodos y constantes para la optimización del flujo de red. Está diseñada para ser implementada por la clase Servidor, lo que asegura que cualquier servidor cumpla con las operaciones necesarias para gestionar y optimizar la red de manera eficiente. Incluye métodos para verificar la administración de proveedores y calcular distancias óptimas para mejorar la intensidad del flujo.

Métodos Principales:

- verificarAdmin (Abstracto): Verifica la existencia de servidores asociados a un proveedor específico dentro del sistema.
- distanciasOptimas (Abstracto): Calcula las distancias óptimas para ubicar servidores para mejorar la intensidad de flujo de red.

Relación con Otras Clases:

- Implementada por Servidor: En la clase Servidor se utiliza esta interfaz para implementar métodos.
- Con Cliente y ProveedorInternet: Colabora con estas clases para analizar las intensidades de flujo y gestionar la red.

4. Descripción de la implementación de características de programación orientada a objetos en el proyecto.

4.1. Clase abstracta y método abstracto: Se sabe que en Python las clases abstractas se simulan mediante metaclasses y decoradores. Esto pasó con la clase Cobertura y 3 de sus métodos.

- Clase: la clase Cobertura proporciona una estructura base que asegura que las subclases implementen funcionalidades clave, promoviendo la reutilización de código y la consistencia. Al ser abstracta, facilita la organización y extensión del código, permitiendo que cada subclase defina su lógica específica de manera controlada.

```
1  from abc import ABC, abstractmethod
2
3  class Cobertura(ABC):
```

Ubicación: gestorAplicacion/enrutadorHFC/cobertura.py, línea 1.

- Método: son implementados por las clases Antena y Router, con diferentes prioridades según la clase y el modelo lógico del programa. El método rastrearGeneracionCompatible es usado principalmente por Antena para la funcionalidad de Test, mientras que intensidadFlujoOptima y intensidadFlujoClientes, se utilizan principalmente en Router para las funcionalidades de Mejora tu Plan y Reporte.

```
12  # //METODOS ABSTRACTOS
13
14  # //METODO UTILIZADO PRINCIPALMENTE POR LA CLASE ANTENA--FUNCIONALIDAD DEL TEST
15  @abstractmethod
16  def rastrearGeneracionCompatible(self, antenasSede, router):
17      pass
18
19  # //METODO UTILIZADO PRINCIPALMENTE POR LA CLASE ROUTER--FUNCIONALIDAD MEJORA TU PLAN
20  @abstractmethod
21  def intensidadFlujoOptima(self, ss, c):
22      pass
23
24  # //METODO UTILIZADO PRINCIPALMENTE POR LA CLASE ROUTER--FUNCIONALIDAD REPORTE
25  @abstractmethod
26  def intensidadFlujoClientes(self, ctes, sevidor, reales):
27      pass
```

Ubicación: gestorAplicacion/enrutadorHFC/cobertura.py, línea 12-27.

Un ejemplo de la implementación sería en la clase Antena:

```
45  # //METODO INSTANCIA---IMPLEMENTACION Y SOBREESCRITURA METODO ABSTRACTO QUE SE HEREDA DE COBERTURA-FUNCIONALIDAD TEST--BUSCA UNA
46  def rastrearGeneracionCompatible(self, antenas_sede, router):
47
48      antenas_cercanas = list(filter(lambda antena: antena.verificarZonaCobertura(router, antena), antenas_sede))
49      antena_encontrada = next(antena for antena in antenas_cercanas if antena.getGeneracion() == router.getGeneracion() or None)
50
51      return antena_encontrada
```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 45.

4.2 Herencia múltiple (“Interfaz”): En Python, un caso que en Java sería una interfaz (clase abstracta con métodos abstractos) se puede lograr mediante herencia múltiple. Se usan clases abstractas con el decorador @abstractmethod, y luego se aplica herencia múltiple para que una clase herede de varias interfaces o clases abstractas.

Se implementó una “interfaz” en Python simulando una clase abstracta llamada Red, que tiene una "constante" estática FLUJO_RED_PRELIMINAR con valor 500, y dos métodos abstractos: verificarAdmin y distanciasOptimas.

```

1  from abc import ABC, abstractmethod
2
3  # CLASE ABSTRACTA
4  class Red(ABC):
5
6      # // "CONSTANTES"
7      FLUJO_RED_PRELIMINAR=500
8
9      # //MÉTODOS
10     @abstractmethod
11     def verificarAdmin(self, proveedores, nombre):
12         pass
13
14     @abstractmethod
15     def distanciasOptimas(self, clientes, listaIntensidadesClientes):
16         pass

```

Ubicación: gestorAplicacion/enrutadorHFC/red.py

La clase Servidor implementa los métodos abstractos de Red, los cuales son clave para ejecutar la funcionalidad 5: Reporte, ya que permite listar y analizar los servidores que pertenecen a un proveedor, facilitando la generación del informe. En cuanto a la constante de la interfaz, está es para el flujo total que todo proveedor de Internet proporciona a cada uno de sus servidores.

```

5  class Servidor(Red):

```

Ubicación: gestorAplicacion/enrutadorHFC/servidor.py, línea 5.

En cuanto a la constante FLUJO_RED_PRELIMINAR, esta alude al flujo total que todo proveedor de Internet proporciona a cada uno de sus servidores.

4.3. Herencia: Tenemos varios casos de herencia que nos ayudan a implementar las funcionalidades de manera eficiente y facilitar el correcto funcionamiento del programa.

- Herencia de la clase Cobertura: La clase Antena y Router heredan de la clase abstracta Cobertura. Esto nos permite aprovechar los atributos y métodos definidos en Cobertura, pues cada uno puede definir sus propias implementaciones de los métodos, para las necesidades de cada clase.

```

8  class Antena(Cobertura):

```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 8.

```

6  class Router(Cobertura):

```

Ubicación: gestorAplicacion/enrutadorHFC/router.py, línea 6.

- Herencia de la clase Router en Conexión: Lo que le permite acceder a todos los atributos y métodos de la clase Router, así conexión puede acceder directamente a los atributos y métodos de Router, cómo la IP, la velocidad de conexión, la cantidad de dispositivos conectados, etc., sin necesidad de crear una instancia de Router por separado.

```
3 class Conexion(Router):
```

Ubicación: gestorAplicacion/enrutadorHFC/conexion.py, línea 3.

4.4. Ligadura dinámica: La ligadura dinámica en Python se cumple de manera implícita a través del mecanismo de herencia y sobrescritura de métodos, similar a Java, pero sin necesidad de marcar explícitamente la sobrecarga de métodos. Cuando se invoca un método en un objeto, Python determina automáticamente qué versión del método debe ejecutarse en función del tipo del objeto en tiempo de ejecución.

- Caso 1: En la clase Cobertura, se define el método intensidadFlujoOptima como abstracto:

```
19 # //METODO UTILIZADO PRINCIPALMENTE POR LA CLASE ROUTER--FUNCIONALIDAD MEJORA TU PLAN
20 @abstractmethod
21 def intensidadFlujoOptima(self,ss,c):
22     pass
```

Ubicación: gestorAplicacion/enrutadorHFC/cobertura.py, línea 19.

Luego, en la clase hija Router, se sobrescribe este método para proporcionar su propia implementación:

```
240 def intensidadFlujoOptima(self, servidoresLocalidad, c):
241
242     from conexion import Conexion
243
244     distancia = int(sqrt(pow(self._servidorAsociado.getCoordenadas().getX() - self._coordenadas.getX(), 2) +
245                          pow(self._servidorAsociado.getCoordenadas().getY() - self._coordenadas.getY(), 2)))
246
247     conexion = Conexion(self._IP, self._down, self._up, self._online, self._generacion, c, c.getModem().getServidorAsociado())
248     velocidadTotal = conexion.getVelocidad()
249
250     distanciaTemporal = 0
251     servidorTemporal = None
252
253     for servidor in servidoresLocalidad:
254
255         distanciaTemporal = int(sqrt(pow(servidor.getCoordenadas().getX() - self._coordenadas.getX(), 2) + pow(servidor.getCoordenadas().getY()
256                                                                 - self._coordenadas.getY(), 2)))
257
258         if(distanciaTemporal <= distancia):
259             distancia = distanciaTemporal
260             servidorTemporal = servidor
261
262     intensidadFlujoMayor = int((servidorTemporal.getFLUJO_RED_NETO()+
263                               (velocidadTotal/len(self._verificarDispositivos(Dispositivo.getDispositivosTotales(), self))) - distancia)*
264                               servidorTemporal.getPORCENTAJE_EFICIENCIA())
265     return [servidorTemporal,intensidadFlujoMayor]
```

Ubicación: gestorAplicacion/enrutadorHFC/router.py, línea 240.

Finalmente, cuando se llame a intensidadFlujoOptima desde una instancia de Router en el código, como en el siguiente fragmento, Python se encarga de invocar automáticamente la versión del método definida en Router, no en Cobertura, ya que el objeto es de tipo Router:

```
165 intensidadMayor = self.intensidadFlujoOptima(servidores, c)
```

Ubicación: gestorAplicacion/enrutadorHFC/router.py, línea 165.

- Caso 2: En la clase Cobertura, se define el método abstracto rastrearGeneracionCompatible.

```
14 # //METODO UTILIZADO PRINCIPALMENTE POR LA CLASE ANTENA--FUNCIONALIDAD DEL TEST
15 @abstractmethod
16 def rastrearGeneracionCompatible(self, antenasSede, router):
17     pass
```

Ubicación: gestorAplicacion/enrutadorHFC/cobertura.py, línea 14.

Luego, el método se sobrescribe en la clase Antena, donde se implementa de manera concreta con el código necesario para realizar la funcionalidad específica de buscar antenas dentro de la zona de cobertura y comparar la generación del router con la de la antena.


```

45 # //METODO INSTANCIA--IMPLEMENTACION Y SOBREESCRITURA METODO ABSTRACTO QUE SE HEREDA DE COBERTURA-FUNCIONALIDAD TEST--BUSCA UNA
46 def rastrearGeneracionCompatible(self, antenas_sede, router):
47
48     antenas_cercanas = list(filter(lambda antena: antena.verificarZonaCobertura(router, antena), antenas_sede))
49     antena_encontrada = next(antena for antena in antenas_cercanas if antena.getGeneracion() == router.getGeneracion() or None)
50
51     return antena_encontrada

```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 45.

Finalmente, el método `rastrearGeneracionCompatible` se invoca sobre un objeto de la clase `Cobertura` (o una de sus subclases como `Antena`) y se realiza la ligadura dinámica para determinar cuál de las implementaciones del método ejecutar. A continuación, se muestra un ejemplo de cómo se invoca este método en el código:

```

1064 antena_nueva=cobertura_actual.rastrearGeneracionCompatible(antenas_sede,self.cliente_.getModem())

```

Ubicación: uiMain/ventanaUsuario.py, línea 1064.

4.5. Serialización y deserialización: Durante las pruebas del código, se identificó un problema relacionado con la serialización de objetos en archivos separados. Se observó que al guardar los objetos en archivos individuales, se generaban referencias específicas para cada objeto y sus atributos, los cuales pertenecían a otras clases.

Por ejemplo, si dos objetos, como 'Servidor' y 'Antena', compartían el mismo objeto 'ProveedorInternet', al serializarlos, se generaban referencias distintas (espacios de memoria diferentes) para el mismo proveedor. Como resultado, al deserializar y mostrar la información de la antena y el servidor, se detectaba que el objeto 'ProveedorInternet' era diferente para cada uno, lo que causaba errores en el funcionamiento del programa.

Para solucionar este problema, se tomó la decisión de serializar únicamente los objetos de las clases 'Antena', 'Dispositivo' y 'Servidor' en archivos independientes. Además, es importante destacar que la serialización no se realiza objeto por objeto, sino que las clases mencionadas anteriormente tienen un atributo estático que es una lista que contiene todos los objetos creados de esa clase. Así, durante la serialización, se guarda este atributo (la lista de objetos de la clase).

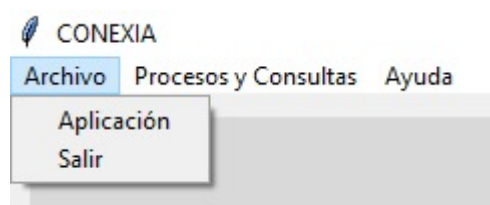
```

24 if __name__ == "__main__":
25
26     # SE DESERIALIZA
27     Deserializador.deserializar()
28
29     # SE AJUSTAN LAS REFERENCIA DE LOS PROVEEDORES
30     ProveedorInternet.proveedores()
31     Antena.antenas()
32
33     # SE CONSTRUYE LA VENTANA DE INICIO
34     ventana = VentanaInicio()

```

Ubicación: uiMain/main.py, línea 24.

En cuanto a la deserialización, se lleva a cabo al ejecutar la aplicación en la clase 'Main', al iniciar el programa. La serialización se realiza cuando el usuario selecciona la opción Archivo - 'Salir' en la ventana principal del programa, ubicada en la barra superior del menú. Esta acción activa el proceso de serialización, mientras que la deserialización ocurre automáticamente al ejecutar la aplicación nuevamente.



4.6. Atributos de clase y métodos de clase: Permiten gestionar datos comunes a todas las instancias

de una clase. Los atributos de clase almacenan información compartida, mientras que los métodos de clase permiten operar sobre esos datos sin necesidad de crear una instancia, mejorando la eficiencia y coherencia del programa.

- **Atributos:** Se utilizan atributos estáticos en las clases Antena, Dispositivo, ProveedorInternet y Servidor, representados como listas que almacenan todos los objetos creados de cada clase. Estos atributos son clave para varios métodos, ya que son comunes a todas las instancias y permiten su uso en métodos estáticos sin necesidad de crear instancias previamente.

```
12  # ATRIBUTO DE CLASE
13  _antenasTotales = []
```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 12.

```
5   # ATRIBUTO ESTÁTICO
6   _dispositivosTotales=[]
```

Ubicación: gestorAplicacion/enrutadorHFC/dispositivo.py, línea 5.

```
15  # ATRIBUTO DE CLASE
16  _proveedoresTotales = []
```

Ubicación: gestorAplicacion/host/proveedorInternet.py, línea 15.

```
9   # ATRIBUTO DE CLASE
10  _servidoresTotales=[]
```

Ubicación: gestorAplicacion/enrutadorHFC/servidor.py, línea 9.

- **Métodos:** Los métodos de clase se definieron con el decorador `@classmethod` y el parámetro 'cls' para acceder a los atributos estáticos de las clases, permitiendo ejecutar procedimientos específicos sin necesidad de instanciar un objeto para llamar al método.

Un ejemplo particular es el método `antenasSede` de la clase `Antena`. Este método busca antenas por sede y está sobrecargado para devolver una lista de antenas o una sola antena, dependiendo de si se incluye el nombre del proveedor.

```
28  # //METODO INSTANCIA---BUSCA LAS ANTENAS QUE ESTAN EN UNA SEDE Y QUE PERTENECEN A UN PROVEEDOR ESPECIFICO
29  @classmethod
30  def antenasSede(cls, sede="", proveedor=None):
31
32      if proveedor == None:
33          antenas = []
34          for antena in cls._antenasTotales:
35              if antena.getSede() == sede:
36                  antenas.append(antena)
37
38          return antenas
39      else:
40          for antena in cls._antenasTotales:
41              if antena.getSede() == sede and antena.getProveedor().getNombre() == proveedor:
42                  return antena
43          return None
```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 28.

4.7. Uso de constante: En Python, aunque no hay soporte explícito para constantes, se sigue la convención de usar mayúsculas para atributos que no deben modificarse.

Un ejemplo de esto es el atributo `ID` en el constructor de la clase, que representa un número de identificación único y no modificable para una persona. Este valor se asigna a través del parámetro `ID` del constructor, como se muestra en el siguiente fragmento de código:

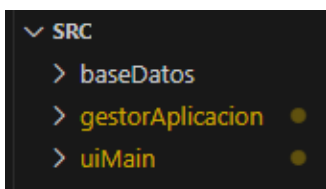
```

19 def __init__(self, nombre=None, ID=0, proveedor=None, modem=None, plan=[], nombrePlan=None, factura=None):
20
21     # ATRIBUTOS
22     self._nombre=nombre
23     self._ID = ID

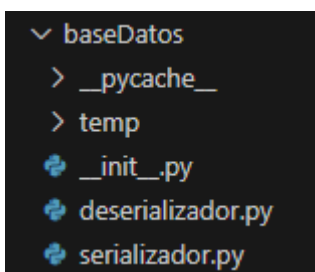
```

Ubicación: gestorAplicacion/host/cliente.py, línea 23.

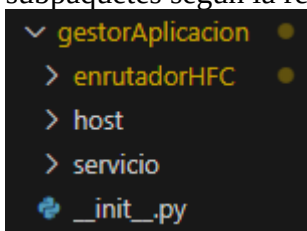
4.8. Encapsulamiento: El sistema está estructurado siguiendo un modelo de 3 capas: persistencia, lógica y presentación.



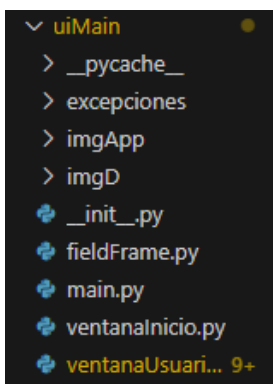
La capa de persistencia se encuentra en la carpeta baseDatos, donde se maneja la serialización de objetos y se mantiene el estado de todos los objetos del sistema. Dentro de esta capa, la carpeta /temp guarda los objetos serializados.



La capa lógica está organizada en el paquete principal gestorAplicación, que se divide en tres subpaquetes según la relación entre las clases: enrutadorHFC, host y servicio.



La capa de presentación, encargada de la interacción del usuario con el sistema, se encuentra en la carpeta uiMain. Las clases de esta capa reciben la información del usuario e imprimen los resultados provenientes del sistema.



Ahora, respecto al nivel de accesibilidad de los atributos de las clases fue definido como 'privado', simulado con el '_' antes de cada atributo, ya que en Python no existe como tal los niveles de accesibilidad, todo es público.

Podemos observar en los atributos de la clase Factura:

```

15      # ATRIBUTOS
16      self._usuario=usuario
17      self._ip=ip
18      self._pagosAtrasados=pagosAtrasados
19      self._precio=precio*pagosAtrasados
20      self._mesActivacion=mesActivacion

```

Ubicación: gestorAplicacion/servicio/factura.py, línea 15.

Finalmente, en cuanto al nivel de accesibilidad de los métodos de las clases, en Python los métodos son siempre públicos. Algunos de ellos son estáticos con el fin de invocarlos desde la clase, más no desde una instancia; todo esto de acuerdo con las necesidades del funcionamiento lógico del programa.

En la clase Antena está el método estático:

```

28      # //METODO INSTANCIA---BUSCA LAS ANTENAS QUE ESTAN EN UNA SEDE Y QUE PERTENECEN A UN PROVEEDOR ESPECIFICO
29      @classmethod
30      def antenasSede(cls, sede="", proveedor=None):
31
32          if proveedor == None:
33              antenas = []
34              for antena in cls._antenasTotales:
35                  if antena.getSede() == sede:
36                      antenas.append(antena)
37
38              return antenas
39          else:
40              for antena in cls._antenasTotales:
41                  if antena.getSede() == sede and antena.getProveedor().getNombre() == proveedor:
42                      return antena
43              return None

```

Ubicación: gestorAplicacion/enrutadorHFC/antena.py, línea 28.

En la clase Servidor está el método de instancia:

```

40      # //IMPLEMENTA EL METODO DE LA CLASE RED--FUNCIONALIDAD REPORTE-
41      def verificarAdmin(self, proveedores, nombre):
42          servidoresProveedor=[]
43          for proveedor in proveedores:
44              if(proveedor.getNombre()==nombre):
45                  for servidor in Servidor._servidoresTotales:
46                      if servidor.getProveedor()!=None:
47                          if(servidor.getProveedor().getNombre()==nombre):
48                              servidoresProveedor.append(servidor)
49
50          return servidoresProveedor

```

Ubicación: gestorAplicacion/enrutadorHFC/servidor.py, línea 40.

4.9. Sobrecarga (1 de métodos y 2 de constructores): En Python no es posible sobrecargar métodos y constructores con diferentes firmas como en Java. En su lugar, se utiliza un solo método o constructor y se aprovechan los valores por defecto en los parámetros para simular la sobrecarga, lo que permite que un solo método o constructor maneje diferentes casos según sea necesario.

- Métodos: Para simular la sobrecarga de métodos, se utilizó un solo método con parámetros por defecto, ajustados según la funcionalidad. En la clase 'ProveedorInternet', el método 'VerificarCliente' se emplea en dos funcionalidades del programa.

```

33     def verificarCliente(self, id_=0, nombre=""):
34         if nombre == "":
35             for cliente in self._clientes:
36                 if cliente.getID()==id_:
37                     if cliente.getNombrePlan()=="PREMIUM":
38                         return cliente
39                     elif cliente.getNombrePlan()=="STANDARD":
40                         return cliente
41                     else:
42                         return None
43             else:
44                 for cliente in self._clientes:
45                     if (cliente.getID() == id_) and (cliente.getNombre() == nombre):
46                         if len(cliente.getModem().verificarDispositivos(Dispositivo.getDispositivosTotales(),
47                             cliente.getModem())) > 2:
48                             return cliente
49                 return None

```

Ubicación: gestorAplicacion/host/proveedorInternet.py, línea 33.

En la primera parte del método, se verifica si el cliente existe y pertenece al proveedor. Recorre una lista de clientes y comprueba el ID y el tipo de plan (PREMIUM o STANDARD). Si se cumplen las condiciones, se retorna al cliente correspondiente. Este comportamiento es esencial para la funcionalidad 2, ‘Visualizar Dispositivos Conectados’.

En el caso contrario, el método comprueba si el cliente tiene el ID y el nombre correctos, obtiene el Router asociado y revisa los dispositivos conectados. Si el número de dispositivos es mayor a dos, retorna el cliente. Este procedimiento es fundamental para funcionalidades 3 y 4.

- **Constructores:** En Python no hay sobrecarga, entonces se implementó la técnica de valores por defecto en los parámetros para simular esta funcionalidad. Esto permite crear instancias de las clases con diferentes combinaciones de atributos, según las necesidades del programa.

El constructor de la clase Factura define varios parámetros como usuario, ip, pagosAtrasados, precio y mesActivacion. Algunos de estos parámetros tienen valores por defecto, lo que permite crear una instancia de la clase con información parcial o completa. Si no se especifican ciertos valores al momento de la creación del objeto, estos toman los valores predeterminados. Por ejemplo, el precio se calcula multiplicando el monto por los pagos atrasados.

Este constructor es utilizado para crear una factura de cliente, con opciones de personalización como los pagos atrasados o el mes de activación.

```

12     # CONSTRUCTOR
13     def __init__(self, usuario=None, ip="", pagosAtrasados=0, precio=0, mesActivacion=None):
14
15         # ATRIBUTOS
16         self._usuario=usuario
17         self._ip=ip
18         self._pagosAtrasados=pagosAtrasados
19         self._precio=precio*pagosAtrasados
20         self._mesActivacion=mesActivacion

```

Ubicación: gestorAplicacion/servicio/factura.py, línea 12.

En el constructor de la clase Servidor, se define un conjunto de parámetros como sede, saturado, proveedor, coordenadas, indice, y pe. Este constructor también utiliza valores por defecto para algunos de los parámetros, permitiendo crear instancias de la clase Servidor con distintos niveles de información. En caso de que se pase un proveedor, se agrega automáticamente la instancia del servidor a la lista de servidores totales.

Este constructor es esencial para crear servidores asociados a un proveedor, con la capacidad de personalizar atributos como la eficiencia de la antena y las coordenadas de ubicación.

```

12  # CONSTRUCTOR
13  def __init__(self, sede="", saturado=False, proveedor=None, coordenadas=None, indice=0, pe=0 ):
14
15      # ATRIBUTOS
16      self._sede=sede
17      self._PORCENTAJE_EFICIENCIA=pe
18      self._saturado=saturado
19      self._proveedor=proveedor
20      self._coordenadas=coordenadas
21      self._routers=[]
22      self._INDICE_SATURACION=indice
23      self._FLUJO_RED_NETO=self._PORCENTAJE_EFICIENCIA*super().FLUJO_RED_PRELIMINAR
24
25      if proveedor!=None :
26          Servidor._servidoresTotales.append(self)

```

Ubicación: gestorAplicacion/enrutadorHFC/servidor.py, línea 12.

4.10. Referencias self (para desambiguar): En Python no existe la palabra clave this, pero su equivalente es self. Al igual que en Java, self se utiliza para desambiguar y hacer referencia al objeto actual de la clase. El programa cuenta con varios ejemplos que utilizan el self para desambiguar, especialmente en los constructores.

Por ejemplo, la clase Plano tiene dos atributos llamados x y y. El constructor recibe dos parámetros con estos mismos nombres. Para hacer referencia a los atributos de la clase y evitar ambigüedad, se utiliza el self, lo que indica que los valores asignados a estos atributos serán los recibidos en los parámetros del constructor.

```

5      # //CONSTRUCTOR
6      def __init__(self,x,y):
7
8          # ATRIBUTOS
9          self._x = x
10         self._y = y
11         self._sede = ""
12         self._zonaCobertura = None

```

Ubicación: gestorAplicacion/servicio/plano.py, línea 5.

En la clase Dispositivo, el constructor también recibe variables con el mismo nombre que algunos de los atributos de la clase. Al igual que en el caso anterior, se usa self para desambiguar y referirse a los atributos de la clase. Además, en este constructor, self también hace referencia al objeto en sí, es decir, al objeto que invoca el constructor, lo que permite añadirlo a la lista Dispositivo._dispositivosTotales, un atributo estático de la clase.

```

8  # CONSTRUCTORES
9  def __init__(self, modem, nombre, generacion):
10
11      # //ATRIBUTOS
12      self._modem=modem
13      self._nombre=nombre
14      self._ipAsociada=modem.getIP()
15      self._generacion=generacion
16      Dispositivo._dispositivosTotales.append(self)

```

Ubicación: gestorAplicacion/enrutadorHFC/dispositivo.py, línea 8.

4.11. Implementación de un caso de enumeración: Se utilizó una enumeración para representar los meses del año a través de la clase Mes.

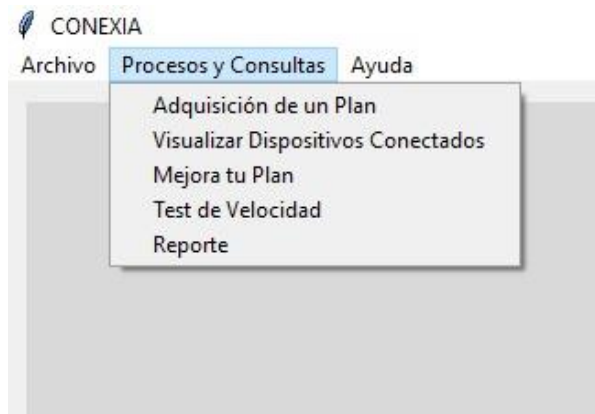
```
5 class Mes(Enum):  
6     ENERO = "ENERO"  
7     FEBRERO = "FEBRERO"  
8     MARZO = "MARZO"  
9     ABRIL = "ABRIL"  
10    MAYO = "MAYO"  
11    JUNIO = "JUNIO"  
12    JULIO = "JULIO"  
13    AGOSTO = "AGOSTO"  
14    SEPTIEMBRE = "SEPTIEMBRE"  
15    OCTUBRE = "OCTUBRE"  
16    NOVIEMBRE = "NOVIMEBRE"  
17    DICIEMBRE = "DICIEMBRE"
```

Ubicación: gestorAplicacion/servicio/mes.py, línea 5.

Esta clase garantiza que el atributo mesActivacion de la clase Factura siempre tenga valores válidos y consistentes, ya que los meses son comunes para todas las facturas y siempre los mismos. La implementación de la enumeración asegura que el atributo mesActivacion sólo pueda tomar uno de los valores predefinidos, lo que mejora la integridad de los datos.

5. Descripción de cada una de las 5 funcionalidades implementadas.

A continuación, se proporcionará una explicación resumida y detallada de todas las funcionalidades implementadas, con el objetivo de comprender su comportamiento y tener una visión completa del contexto en el que se desarrollan.



Antes de profundizar en el cuerpo completo de las funcionalidades: Adquisición de un plan, Visualización de dispositivos conectados, Mejora de un plan, Prueba de velocidad de Internet (test), Informe o reporte de anomalías, es importante brindar claridad sobre ciertos aspectos que se utilizarán de manera recurrente en estas funcionalidades. Estos son fundamentales para el correcto funcionamiento de las acciones que se llevarán a cabo en cada una de ellas.

En el programa, los datos se introducen a través de una interfaz gráfica. Para ello, se utilizó el paquete Tkinter en Python, que facilita el desarrollo de interfaces gráficas de usuario (GUI). Para recolectar los datos correspondientes, se empleó un objeto de tipo FieldFrame. A continuación se muestra el código utilizado para construir esta interfaz y obtener los datos del usuario:

```

759 # CONSTRUCCION FIELDFRAME
760 criterios = ["Tipo de Usuario", "Documento", "Proveedor"]
761 valores=["\tCliente", "", ""]
762
763 frameInterior=tk.Frame(frameInformacion, bg="gray95", height=100, width=300)
764 frameInterior.pack( expand=True, fill="none", padx=100, pady=10, anchor="n")
765
766 fp = FieldFrame(frameInterior, "Datos", criterios, "Valor", valores)
767 fp.configure(borderwidth=0, bg="gray95",)
768 fp.pack(fill="y", pady=3, padx=5)

```

Ubicación: uiMain/ventanaUsuario, línea 759.

La interfaz presenta los criterios de entrada de datos como "Tipo de Usuario", "Documento", y "Proveedor". Los valores que el usuario introduce se corresponden con estos criterios y, por ejemplo, para el tipo de usuario se elige "Cliente". Los datos de entrada que se reciben a través de esta interfaz son necesarios para la funcionalidad "Visualizar Dispositivos Conectados", que luego se procesan para realizar las operaciones correspondientes.

Datos solicitados	Valor
Tipo de Usuario	Cliente
Nombre	
Documento	
Sede	

Aceptar Borrar

Existen más posibilidades para los criterios de entrada de datos en la interfaz para que los usuarios puedan ingresar la información necesaria para completar la funcionalidad del programa. Por ejemplo, en Mejora de un plan, se solicita el nombre de usuario.

5.1. Funcionalidad 1: Adquisición de un plan.

La funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia. En ella interactúan objetos de clases como 'Cliente', 'Factura' y 'ProveedorInternet' que mediante diferentes métodos se relacionan con objetos tipo 'Router', 'Servidor' y 'Antena'.

En principio, se le solicita al usuario ingresar sus datos: nombre, número de identificación (documento) y sede; cada uno con su respectivo mensaje. Así en la interfaz:

CONEXIA

Archivo Procesos y Consultas Ayuda

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

Datos solicitados	Valor
Tipo de Usuario	Cliente
Nombre	Valeria
Documento	1021
Sede	B

Aceptar Borrar

Posteriormente, se verifica que la sede ingresada exista (A,B,D,C), en caso de que el usuario ingrese el dato de manera incorrecta, se lanzará una excepción. Es importante aclarar que introducir un dato en minúscula o mayúscula no supone un problema. Una vez se valide la sede, se procede a listar y mostrar con la interfaz, los proveedores disponibles en dicha localidad, para posteriormente solicitar al usuario que ingrese el número de la opción que desea elegir.

Para conocer los proveedores de una sede específica, se llama al método estático 'proveedorSede' de la clase 'ProveedorInternet', al cuál se le pasa por parámetro la sede (String) que introdujo el usuario. Este método se encarga de recorrer con un 'for' el atributo estático 'servidoresTotales' de la clase 'Servidor', comparando si el atributo 'sede' del servidor de cada iteración es igual al ingresado por parámetro, en caso de que esto suceda, se añade el objeto 'ProveedorInternet' asociado a una lista que contiene objetos de este tipo.

```

75     # //METODO ESTÁTICO--FUNCIONALIDAD ADQUISICION DE PLAN--
76     @classmethod
77     def proveedorSede(cls,sede):
78         proveedores=[]
79         for servidor in Servidor.getServidoresTotales():
80             if servidor.getSede() == sede:
81                 proveedores.append(servidor.getProveedor())
82
83         return proveedores

```

Ubicación: gestorAplicacion/host/proveedorInternet, línea 75.

Nota: El uso de interfaces gráficas en Python permite evitar la necesidad de utilizar bucles while para reintentar un proceso que haya fallado. En lugar de eso, simplemente se regresa al objeto FieldFrame, que vuelve a solicitar los datos necesarios para continuar con el flujo del programa.

Luego de esto, se comprueba que el número ingresado sea válido, en caso contrario, se lanza una excepción. Cuando el usuario introduzca una opción correcta, se asigna a la variable 'proveedorActual'. Después de esto se declara un apuntador llamado 'clienteActual' y se le asigna el cliente que retorna el método 'crearCliente'.

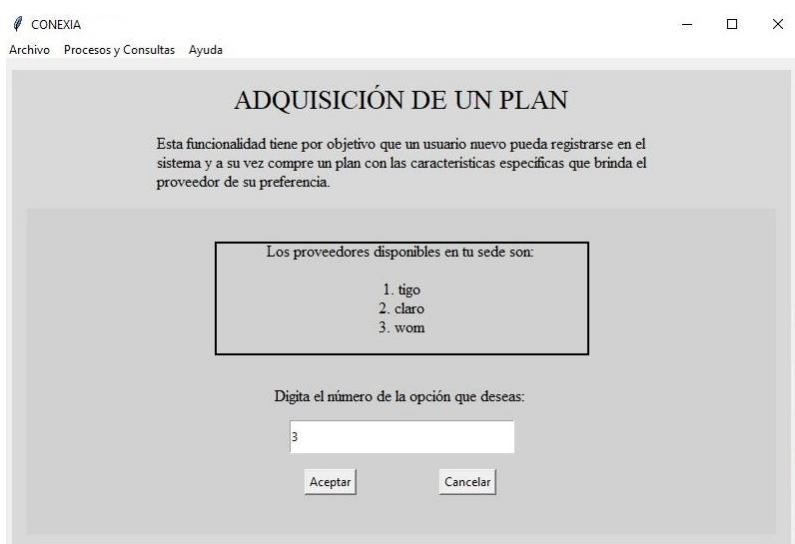
```

334     # SE GUARDAN LOS OBJETOS
335     proveedorActual = proveedoresSede[opcionEscogida-1]
336     clienteActual = proveedorActual.crearCliente(nombre,documento)

```

Ubicación: uiMain/ventanaUsuario, línea 334.

Este método recibe el nombre y el ID ingresado por el usuario, y como su nombre lo indica se encarga de instanciar un objeto 'Cliente' con los datos pasados por parámetro.



Se procede a verificar que el proveedor elegido tenga cupos disponibles, para este fin se obtiene el tamaño de la lista 'clientes' del proveedor (atributo del objeto) y compara si este valor es menor al número de clientes máximo que puede tener el proveedor, el cuál también es un atributo de dicho objeto. En caso contrario, se le indica al cliente que no hay cupos y lo devuelve al menú inicial. Cuando hay cupos, se muestra en la ventana, los planes disponibles del proveedor con sus respectivas características.

Para obtener esto, se llama con el apuntador 'proveedorActual', al método 'planesDisponibles' (método principal), el cual recibe el objeto Cliente previamente instanciado y retorna un lista de tipo String.

```

345     planes = proveedorActual.planesDisponibles(clienteActual)

```

Ubicación: uiMain/ventanaUsuario, línea 345.

Este método se encarga de verificar que la cantidad de clientes por tipo de plan sea menor al máximo que puede haber en el plan correspondiente. Así es el cuerpo:

```

50  # //METODOO INSTANCIA-- FUNCIONALIDAD ADQUISICION DE PLAN-- OBTIENE LOS
51  def planesDisponibles(self, cliente):
52
53      clientes_basic=cliente.buscarCliente(self._clientes,"BASIC")
54      clientes_standard=cliente.buscarCliente(self._clientes,"STANDARD")
55      clientes_premium=cliente.buscarCliente(self._clientes,"PREMIUM")
56      planes_disponibles=[]
57
58      if len(clientes_basic) < self._CLIENTES_BASIC:
59          planes_disponibles.append("BASIC")
60
61      if len(clientes_standard) < self._CLIENTES_STANDARD:
62          planes_disponibles.append("STANDARD")
63
64      if len(clientes_premium) < self._CLIENTES_PREMIUM:
65          planes_disponibles.append("PREMIUM")
66
67      return planes_disponibles

```

Ubicación: gestorAplicacion/host/proveedorInternet, línea 56.

Nota: recordar que los objetos 'ProveedorInternet' cuentan con tres atributos que indican la cantidad máxima de clientes que pueden tener en cada uno de sus planes.

Para llevar a cabo esto, se necesita obtener una lista que contenga los clientes sesgados por tipo de plan, por ende, se hace el llamado al método 'buscarCliente' que recibe el atributo 'clientes' del objeto 'ProveedorInternet' (el objeto que está ejecutando el método principal) y un String con el nombre del plan del cuál se desea listar los clientes. Para poder llamar este último método se necesita un objeto Cliente, dado que no es estático; esta es la razón por la que al método principal se le pasa un objeto tipo Cliente.

De esta manera, se invoca el método 'buscarCliente' tres veces porque hay tres tipos de plan: Basic, Standard, Premium; dicho método retorna la lista con los clientes del proveedor y del tipo de plan indicado, es decir, se tienen tres arreglos, de los cuales se obtiene el tamaño de estos y se compara si es menor al máximo de clientes que puede haber en dicho plan, en caso de que esto sea afirmativo, se añade el nombre de este plan a un arreglo de tipo String que contiene los planes disponibles. Al final se retorna esta lista, en la clase 'ventanaUsuario' se recorre y se obtienen las características del servicio para que el usuario las visualice en la ventana respectiva.

Después de esto se le solicita al cliente que ingrese el nombre del plan que desea adquirir; una vez se realice esto, se procede a validar que el nombre ingresado sea correcto y que efectivamente sea un plan que esté disponible. Es indiferente si el usuario digita el nombre en mayúscula o minúscula, dado que el sistema mismo se encarga de pasar todo a mayúscula, no obstante sí es importante que digite el nombre tal cual es. Cuando la información obtenida es correcta, se asigna a una variable llamada 'planEscogido', el nombre del plan que el cliente indicó. En caso contrario, se lanza una excepción. Cuando se pasa el filtro anterior, se continúa pidiéndole al usuario que digite las coordenadas de su ubicación.

CONEXIA
Archivo Procesos y Consultas Ayuda

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

Digita las coordenadas de tu ubicación, estas tienen un límite de acuerdo con tu sede.

Límite Inferior en X: 0
Límite Superior en X: 40
Límite Inferior en Y: 60
Límite Superior en Y: 100

Datos solicitados	Valor
Tipo de Usuario	Cliente
Coordenada en X	0
Coordenada en Y	100

De acuerdo con la sede que la persona haya indicado al principio de la funcionalidad, estas coordenadas deberán estar dentro de un rango, que es delimitado por el cuadrado que corresponde a dicha sede. Por tanto, los datos que el cliente digite serán validados según los límites establecidos en el esquema antes expuesto; cuando se ingrese una coordenada fuera del rango, se lanza una excepción.

```

273 # SE CONSTRUYE EL FIELDFRAME QUE SOLICITA LOS DATOS CORRESPONDIENTES
274 criterios = ["Tipo de Usuario", "Coordenada en X", "Coordenada en Y"]
275 fp =FieldFrame(frameInformacion, criterios, "Valor", ["Cliente", "", ""])
276 fp.configure(borderwidth=1, relief="groove",bg="gray82")
277 fp.place(relx=0.15, rely=0.45, relheight=0.5)

```

Ubicación: uiMain/ventanaUsuario, línea 273.

Después se hace el llamado al método 'configurarPlan' con el objeto 'Cliente' que está realizando todo el proceso; a este método se le pasa el planEscogido (String), clienteActual (Cliente), sede (String), coordenadaX (int) y coordenadaY (int). Este es otro método principal de la funcionalidad, ya que se encarga de asignar todas las características al cliente, así como de crear y asignar un objeto 'Router', asociarlo al objeto 'Servidor' correspondiente (que sea de la sede y proveedor adecuado) y al objeto 'Antena' al cuál estará conectado. Al final retorna un objeto de la clase 'Factura', que para este momento sólo tiene asignado el cliente mismo. El apuntador a este objeto es 'facturaCliente'.

```

198 # SE CONFIGURA EL PLAN Y SE GENERA UNA FACTURA
199 facturaCliente = clienteActual.configurarPlan(planEscogido, clienteActual, proveedorActual, sede, coordenadaX, coordenadaY)

```

Ubicación: uiMain/ventanaUsuario, línea 198.

Luego se invoca el método 'generarFactura' mediante el objeto 'Factura' obtenido anteriormente. A este método se le pasa un objeto 'Cliente' y un objeto 'ProveedorInternet'. Dentro de él, se configura la factura del usuario de acuerdo con el plan y el proveedor, además se le asigna esta nueva factura al cliente que se pasa por parámetro y retorna el objeto 'Factura' con estos cambios.

```

204 # SE PROCEDE A CONFIGURAR LA FACTURA
205 facturaCliente = facturaCliente.generarFactura(clienteActual,proveedorActual)

```

Ubicación: uiMain/ventanaUsuario, línea 204.

Aquí se muestra en pantalla el objeto 'Factura' obtenido anteriormente, presentando la información del cliente, la IP del router instanciado y demás información importante, así como el precio a pagar por el servicio. Posteriormente, se le solicita al usuario que digite la cantidad de dinero que debe cancelar, por ende, para comprobar que la cantidad ingresada es la correcta se llama al método 'accionesPagos' de la clase 'Factura', por medio del objeto 'Factura' del cliente y se le pasa

un 2 por parámetro. Este método verifica que lo que se ingrese sea exactamente igual al precio de la factura (atributo) y a continuación se presenta el fragmento en donde se pasa 2 por parámetro.

```

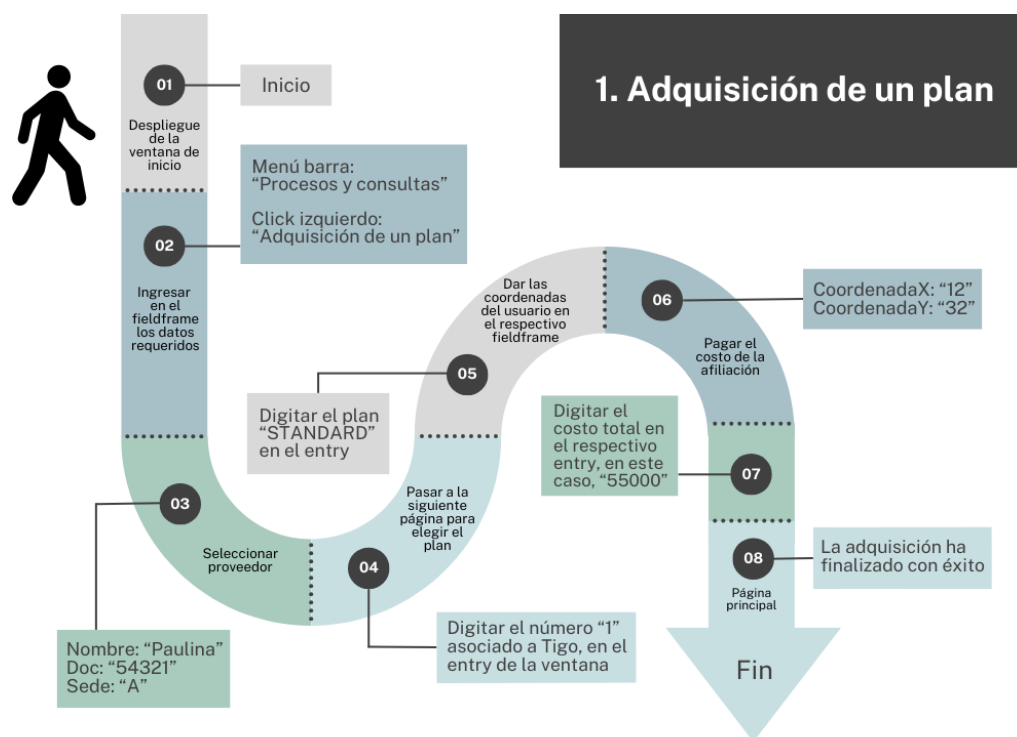
45     elif opcElegida==2:
46         abonoUsuario = int(abonoUsuario)
47         if abonoUsuario==self._precio:
48             self._precio=0
49             self._pagosAtrasados=0
50             return self._usuario.getModem()
51         else:
52             raise ErrorPagoTotal()
53     return None

```

Ubicación: gestorAplicacion/servicio/factura, línea 45.

Si el usuario ingresa un valor menor o mayor al indicado, se llamará a la excepción `ErrorPagoTotal` indicando que se debe ingresar la totalidad del precio. Cuando el usuario realice el proceso de pago correctamente, se le indicará por medio de un mensaje que su adquisición del plan ha finalizado con éxito y el sistema automáticamente lo devolverá a la pantalla principal, indicando que la funcionalidad ha terminado.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad, mediante un caso de prueba.



5.2. Funcionalidad 2: Visualizar los dispositivos conectados.

La funcionalidad es la encargada de mostrar por pantalla la lista de dispositivos, que se encuentran vinculados a un router asociado a un cliente, presentando por consola algunas de las características básicas que posee cada dispositivo, cabe aclarar que esta funcionalidad está disponible únicamente para clientes con planes tipo 'Standard' o 'Premium' y además si el cliente que desee ejecutar dicha funcionalidad tiene más de dos pagos atrasados se brindan posibilidades para saldar o abonar a la deuda, pues para clientes con esta condición la funcionalidad no estará disponible hasta que solo deban 2 o menos pagos. En la implementación de esta funcionalidad intervienen diferentes objetos de las clases 'Dispositivo', 'Router', 'Factura' y 'Cliente'.

Inicialmente, se solicita al usuario su identificación y el nombre de su proveedor, esto para verificar si en efecto el usuario en cuestión se encuentra registrado en los datos almacenados por el

proveedor que se digite.

```

759     # CONSTRUCCION FIELDFRAME
760     criterios = ["Tipo de Usuario", "Documento", "Proveedor"]
761     valores=["\tCliente", "", ""]
762
763     frameInterior=tk.Frame(frameInformacion, bg="gray95", height=100, width=300)
764     frameInterior.pack( expand=True, fill="none", padx=100, pady=10, anchor="n")
765
766     fp = FieldFrame(frameInterior, "Datos", criterios, "Valor", valores)
767     fp.configure(borderwidth=0,bg="gray95",)
768     fp.pack(fill="y",pady=3, padx=5)

```

Ubicación: uiMain/ventanaUsuario, línea 759.

Posteriormente, se obtiene el objeto 'ProveedorInternet' y se realiza un llamado al método 'verificarCliente' en dicha clase, en donde se pasa el valor del 'ID' del usuario y evalúa el tipo de plan que posee, pues en caso de tener un plan tipo 'Basic' no es posible acceder a esta función, en tal caso se lanza una excepción.

Además, comprueba que el cliente se encuentre en su lista de 'clientes' (atributo), para ello lo recorre y compara los ID. Si todo es válido, retornará el objeto 'Cliente' que se usará en otros métodos de la funcionalidad.

```

43     else:
44         for cliente in self._clientes:
45             if (cliente.getID() == id_) and (cliente.getNombre() == nombre):
46                 if len(cliente.getModem().verificarDispositivos(Dispositivo.getDispositivosTotales(), cliente.getModem())) > 2:
47                     return cliente
48         return None

```

Ubicación: gestorAplicacion/host/proveedorInternet, línea 43.

Si el método retorna 'null', quiere decir que el cliente no se encuentra en la base de datos del proveedor ingresado.

En el siguiente paso se verifica si el usuario tiene pagos atrasados por medio del método 'verificarFactura' que recibe el objeto 'Factura' del cliente y es ejecutado desde el mismo. Este retorna 'false' si efectivamente tiene más de 2 pagos atrasados, y 'true' en caso contrario.

```

738     if clienteActual.getFactura().verificarFactura(clienteActual.getFactura()) == False:

```

Ubicación: uiMain/ventanaUsuario, línea 738.

Si tiene más de 2 pagos atrasados, por medio del objeto 'Factura' asociado al cliente, se realiza un llamado al método 'accionesPagos' de la clase 'Factura', en este método se brindan las posibilidades de pago: saldar completamente la deuda o simplemente abonar a ésta. Adicionalmente se muestra por pantalla el valor de su deuda y el número de pagos atrasados que tiene. Una vez se elija la opción deseada, se pasa como parámetro a este método y luego de realizar los respectivos cálculos relacionados con los valores a pagar o abonar, se verifica nuevamente si luego de este proceso se cumple con la condición de tener 2 o menos pagos atrasados para poder acceder y completar la funcionalidad, si se cumple esta condición, el método retornará el objeto 'Router' del cliente, pero en caso de no ser así el método retornará None y el sistema volverá al menú principal.

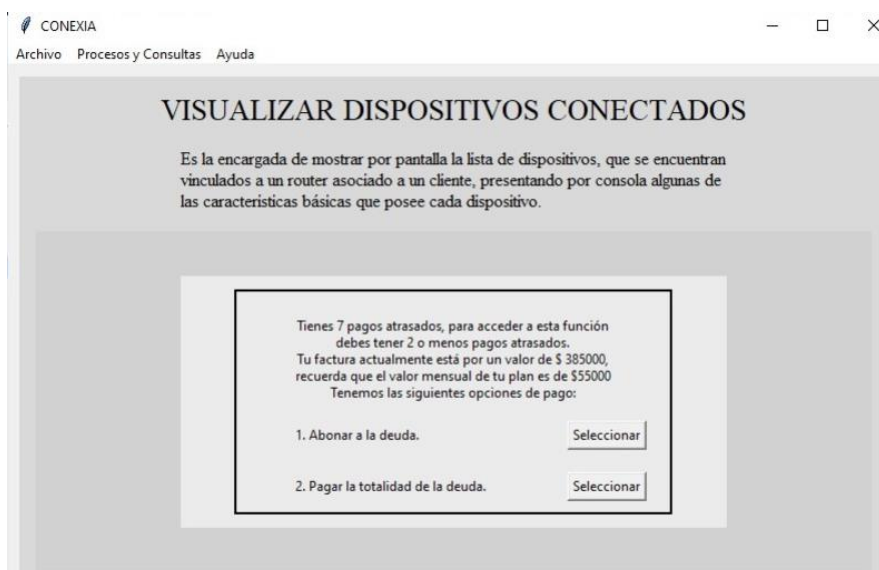
```

696     # Segundo Metodo Funcionalidad
697     if (clienteActual.getFactura().accionesPagos(2,int(entry.get())) != None):
698         mostrarResultado()

```

Ubicación: uiMain/ventanaUsuario, línea 696.

Así se ve en la interfaz:

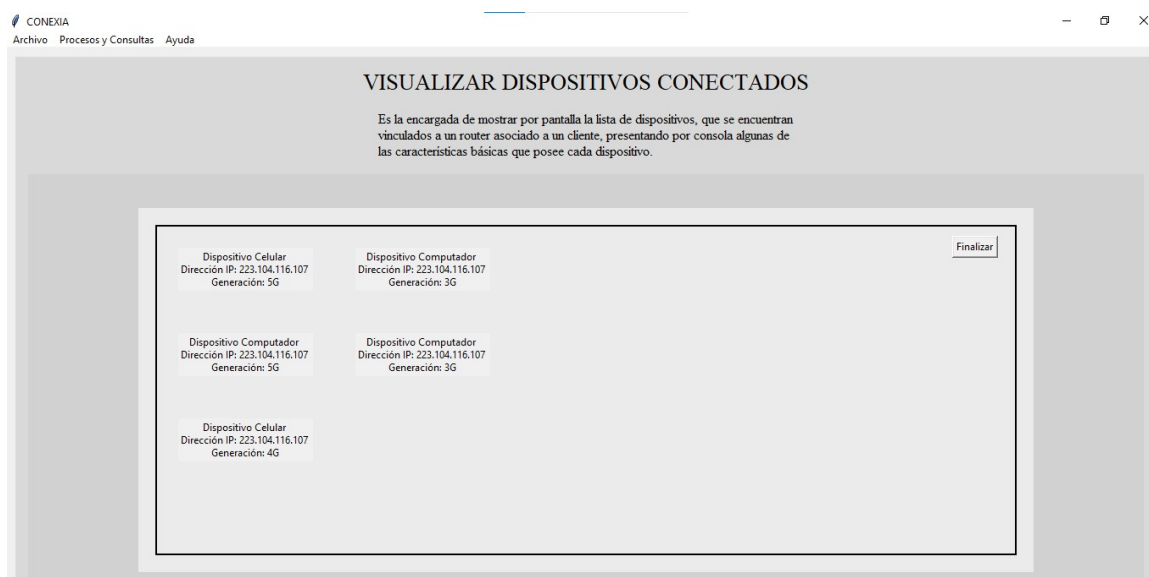


Una vez se verifique la condición de pagos atrasados, por medio del objeto 'Router' asociado al cliente se realiza el llamado al método 'verificarDispositivos' en la clase 'Router' que recibe como parámetros, una lista donde está contenido el total de objetos 'Dispositivo' de todos los routers, este se encuentra como atributo estático en la clase 'Dispositivo', y también se le pasa como parámetro el objeto 'Router' vinculado al cliente. En este método inicialmente se crea una lista donde se almacenarán los dispositivos conectados al router (elemento que retornará el método una vez finalice). Lo anterior se hace recorriendo la lista de dispositivos totales y tomando como medio de comparación la ip asociada (atributo) al dispositivo de la iteración con la ip asociada (atributo) al router del cliente y en caso de coincidir se añade a la lista.

```
560 # PRESENTACIÓN RESULTADOS
561 lista = clienteActual.getModem().verificarDispositivos(Dispositivo.getDispositivosTotales(), clienteActual.getModem())
```

Ubicación: uiMain/ventanaUsuario, línea 560.

Finalmente al ejecutar los métodos anteriores se muestra por pantalla las opciones de visualización que se tienen, estas con relación a un atributo definido como generación en la clase 'Dispositivo'. Luego de elegir la opción deseada y validar que corresponde a una de las opciones permitidas, se procede a mostrar los resultados por pantalla y de esta manera finaliza la ejecución de la funcionalidad.



A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad, mediante un caso de prueba.



5.3. Funcionalidad 3: Mejora tu plan.

La funcionalidad se propone brindando una recomendación con base a las características del plan actual que posee el cliente, pues estas se tienen en cuenta para realizar ciertos cálculos donde intervienen objetos y atributos de diferentes clases como 'Servidor', 'Cliente', 'Conexión' y 'Router', en los que se comparan distancias, intensidades de flujo y precios permitiendo así encontrar la mejor opción posible de plan. De este modo, se recomienda un cambio al cliente dándole la posibilidad de acceder a beneficios que puedan ser más provechosos para él.

Esta funcionalidad se encuentra disponible para usuarios que ya estén registrados y hayan adquirido un plan, cabe resaltar que la recomendación y el cambio sólo se podrá realizar para el mismo tipo de plan del cliente (es decir si tiene basic se podrá cambiar a un plan de tipo basic). La funcionalidad nombrada es ideal para usuarios que deseen adquirir un mejor plan y cuenten con al menos 3 dispositivos conectados a su red.

En principio, se solicita al usuario datos básicos para realizar las verificaciones correspondientes a la existencia de un registro del usuario en cuestión, teniendo en cuenta los datos ingresados y pasándolos por parámetros al método 'verificarCliente' de la clase 'ProveedorInternet'. El retorno del método será el objeto 'Cliente', si toda la información es correcta, en un caso contrario se lanzará una excepción.

```
886 clienteActual = proveedorActual.verificarCliente( documento, nombre)
```

Ubicación: uiMain/ventanaUsuario, línea 886.

Seguidamente, por medio de la clase 'Servidor' se procede a realizar un llamado al método estático 'buscarServidores' que recibe como parámetro un 'String' con la localidad (atributo) en la que se encuentra el objeto 'Router' asociado al usuario. En este método se crea un arreglo en el que se almacenarán los servidores (objetos) que coincidan con la localidad introducida como parámetro; lo anterior se verifica recorriendo la lista de servidores totales (atributo estático de la clase Servidor) y además de esta condición también se debe cumplir que el servidor no se encuentre en estado 'saturado' (atributo). Por último el tipo de retorno del método es la lista de servidores en dicha sede o localidad (este elemento será de utilidad en el siguiente método).


```

910  localidad = clienteActual.getModem().getSede() #Sede del cliente
911  listaServLoc = Servidor.buscarServidores(localidad) #Segundo metodo - Total servidores de la localidad

```

En un tercer momento se invoca al método 'protocoloSeleccionServidor' de la clase 'Router' utilizando el módem (objeto) que se encuentra asociado al usuario, pasando como parámetros la lista de servidores disponibles en la sede (obtenido con el método antes explicado) y el objeto 'Cliente'. El método en cuestión, inicialmente obtiene las coordenadas (atributo) del router, obtiene el arreglo de dispositivos conectados a este por medio del método mencionado en la funcionalidad Visualizar Dispositivos Conectados ('verificarDispositivos'), seguidamente se calcula la distancia actual entre el router y su servidor asociado; además se obtiene la velocidad total de conexión a través de un objeto 'Conexión' y posteriormente se realiza el cálculo de la intensidad de flujo por medio de una fórmula ya planteada en la que intervienen algunos de los elementos mencionados anteriormente o ciertos atributos de estos.

El paso siguiente en la ejecución del método 'protocoloSeleccionServidor' (método principal) corresponde a realizar el llamado a otro método ('intensidadFlujoOptima') que permite estimar el servidor, que en la sede que se encuentra el usuario, posea la mejor intensidad de flujo y esté más cercano, esto realizando el mismo procedimiento utilizado anteriormente pero para todos los servidores de la localidad. El método recibe la lista de servidores de la sede que recibe el método principal, además del objeto 'Cliente'. El retorno es una lista que contiene el objeto 'Servidor' y el valor de la intensidad de flujo mayor.

Una vez se obtienen estos valores se comparan con lo que actualmente posee el usuario, teniendo en cuenta los criterios de verificación en los posibles casos:

1. Misma intensidad de flujo y mismo servidor: en este caso al usuario no se le proporciona una alternativa de cambio dado que su plan es el ideal dentro de sus posibilidades.
2. Misma intensidad de flujo y diferente servidor: se evalúa según el tipo de plan que tenga el usuario, pues el cambio está permitido únicamente para un mismo tipo de plan; una vez se verifica esta condición se compara también el precio del plan recomendado con el del usuario para brindarle la mejor alternativa (mejores condiciones a un precio más favorable). Se añade en una lista las características: intensidad de flujo actual, la que se encontró como mejor opción con su respectivo servidor, el proveedor y las características que describen el plan que se tiene como mejor y esta lista es justamente el retorno del método 'protocoloSeleccionServidor'.
3. Mayor intensidad de flujo en el servidor recomendado: se realiza lo mismo que en el caso anterior, sin embargo, no se comparan los precios de los planes, dado que la intensidad de flujo del servidor recomendado ya es mayor que la actual. El retorno es una lista con los mismos elementos expuestos en el caso 2.

```

912  mejoraIdeal = clienteActual.getModem().protocoloSeleccionServidor(listaServLoc, clienteActual) #Tercer metodo
913

```

Ubicación: uiMain/ventanaUsuario, línea 912.

Una vez se obtienen los resultados, se muestran al usuario junto con las características de su plan actual para que se puedan comparar y se brinda la posibilidad de realizar el cambio, sin embargo antes de hacerlo se verifica la cantidad de pagos atrasados que tenga, pues en caso de tenerlos se redirecciona a realizar el pago completo de la deuda para ejecutar exitosamente el cambio (de nuevo con el método accionesPagos(opcion2)). Una vez cancelada, se le setean los cambios a los objetos respectivos, dando fin a la funcionalidad.

CONEXIA
Archivo Procesos y Consultas Ayuda

MEJORA TU PLAN

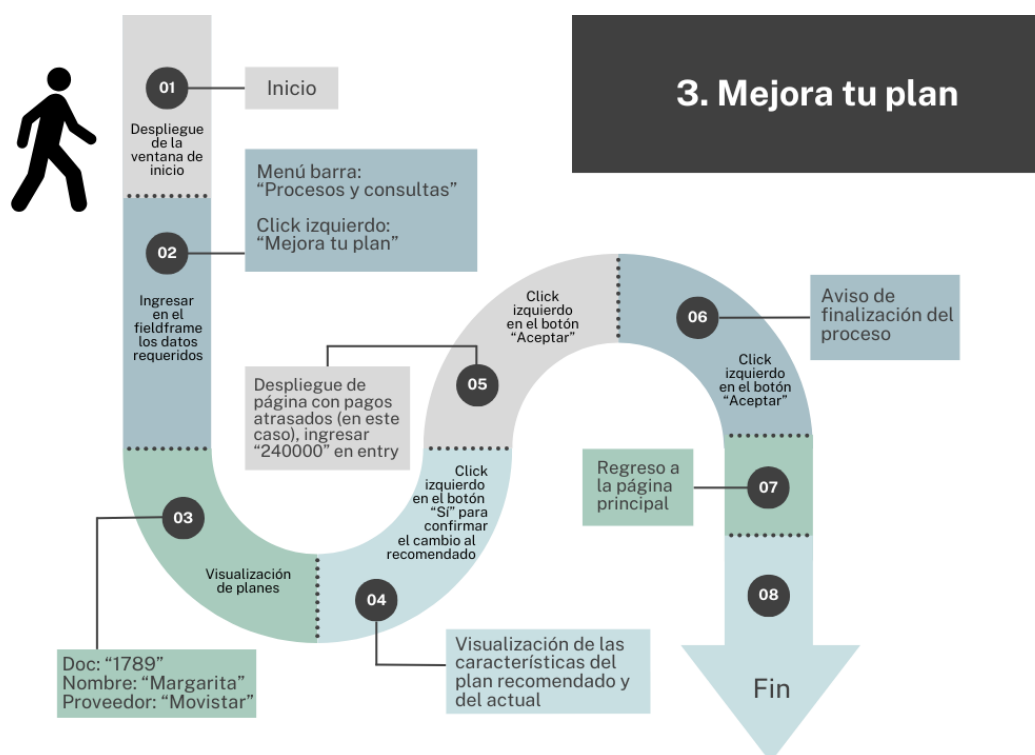
Es la encargada de verificar si el usuario tiene posibilidad de mejorar su plan. Caso tal de que se pueda y el usuario quiere cambiarse a ese plan sugerido, hara todo el proceso para cambiar el plan anterior por el nuevo.

Al realizar las verificaciones encontramos que este nuevo servicio puede ser mejor para ti.

Te presentamos la siguiente recomendación:	Tu plan actual tiene las siguientes características:
Intensidad de flujo: 515	Intensidad de flujo: 497
Proveedor: movistar	Proveedor: tigo
Plan: STANDARD	Plan: STANDARD
Megas de subida: 28	Megas de subida: 30
Megas de bajada: 70	Megas de bajada: 50
Precio: 60000	Precio: 55000

¿Deseas cambiar tu plan por la recomendación dada?

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad, mediante un caso de prueba.



5.4. Funcionalidad 4: Test.

La funcionalidad tiene como objetivo facilitar a los usuarios la realización de pruebas de velocidad de su conexión de red, permitiéndoles verificar la velocidad de carga y descarga de datos, así como el tiempo de respuesta o ping en milisegundos. Esto les brinda la posibilidad de evaluar el estado de su red y, además, recibir recomendaciones para optimizarla y evitar problemas futuros que puedan afectar el rendimiento del sistema.

También se proporcionará información sobre el estado de conexión de la antena, incluyendo detalles sobre la cobertura y generación de la señal, con el objetivo de mantener al usuario informado sobre las características específicas de su zona de cobertura. En esta funcionalidad intervienen objetos tipo 'Antena', 'Router', 'Cliente' y 'ProveedorInternet'.

Para comenzar el respectivo Test, es fundamental verificar si el usuario se encuentra registrado en la base de datos de algún proveedor. Con este fin, se solicita al usuario proporcionar cierta información básica que será utilizada para implementar métodos de verificación. Estos métodos

permitirán obtener la información necesaria para que el Test funcione de manera adecuada y precisa.

```
981     criterios = ["Tipo de Usuario", "ID", "Proveedor"]
```

Ubicación: uiMain/ventanaUsuario, línea 981.

Luego, se invoca el método 'verificarCliente' mediante el apuntador 'proveedorActual'. Este método tiene como finalidad verificar si los datos proporcionados por el usuario se encuentran registrados como un cliente en la base de datos del proveedor correspondiente.

```
886     clienteActual = proveedorActual.verificarCliente( documento, nombre)
```

Ubicación: uiMain/ventanaUsuario, línea 886.

En caso de que el cliente exista en la base de datos, el método devolverá un objeto de tipo 'Cliente', el cual se asignará al apuntador 'clienteActual'. Por otro lado, si el cliente no se encuentra en la base de datos, el método retornará un valor None. En este caso, se requerirá al usuario que proporcione nuevamente datos válidos para proceder con la operación, mediante una excepción..

Después, se declara una variable de tipo llamada 'resultado'. Aquí se invoca otro de los métodos principales de la funcionalidad denominado 'test', el cual se encuentra en la clase 'Router' y es ejecutado por el objeto 'Router' asociado al cliente. Este método recibe como argumento un objeto de tipo 'Cliente' que corresponde al 'clienteActual'. A partir de este objeto, se extrae información relevante relacionada con las velocidades de subida y bajada de datos, así como la cantidad de dispositivos conectados, entre otros. Con base en estos datos, se realizan ciertos cálculos y se genera una cadena de texto que representa la conclusión del test(lo que retorna el método), es decir, el estado de la red.

```
1096     resultado=self.cliente_.getModem().test(self.cliente_)
```

Ubicación: uiMain/ventanaUsuario, línea 1096.

Además, se instancia una variable llamada 'antenas_sede'. En esta variable, se utiliza un método estático importante de la funcionalidad llamado 'antenasSede', el cual pertenece a la clase 'Antena'. Este método recibe como argumento un 'String' que indica la sede o localidad donde se encuentra ubicado el 'clienteActual', y devuelve la lista de todas las antenas (objetos) ubicadas en esa sede. La información necesaria para este argumento se obtiene a través de este objeto tipo 'Cliente'.

```
1097     antenas_sede=Antena.antenasSede(self.cliente_.getModem().getSede())
```

Ubicación: uiMain/ventanaUsuario, línea 1097.

Estas acciones adicionales permiten obtener información detallada y sumamente importante para la correcta ejecución del programa, sobre el estado de la red del cliente y, al mismo tiempo, acceder a la lista de antenas disponibles en la sede o localidad correspondiente al cliente.

Con base en la conclusión obtenida en la variable 'resultado', se llevará a cabo una verificación utilizando condicionales 'if' evaluados en diferentes escenarios para sugerir y proponer la mejor solución en relación al estado actual de la red del usuario.

- Primer caso: Si el resultado indica que la red está saturada, es decir, que la cantidad de dispositivos asociados a la red del 'clienteActual' supera la capacidad del plan adquirido.

```

1104 if "saturada" in resultado:
1105
1106     messagebox.showinfo(title = "Saturación en Red",
1107         message = resultado,
1108         detail = "Se recomienda remover dispositivos")
1109
1110
1111     text_disp = tk.Label(self.frame_display, text="Dispositivos:")
1112     text_disp.grid(row=0, column=0, padx=50, pady=10, sticky="w")
1113
1114
1115     for i, dispositivo in enumerate(dispositivos_cliente):
1116         text_disps = tk.Label(self.frame_display, text=dispositivo)
1117         text_disps.grid(row=i+1, column=0, padx=50, pady=10, sticky="w")

```

Ubicación: uiMain/ventanaUsuario, línea 1104.

Como solución, se sugiere al usuario que elimine ciertos dispositivos (tiene un cuadro para seleccionar cuál) y se muestra una lista en forma de columna con todos los dispositivos asociados al usuario, para que pueda identificar cuáles desea desconectar. Esta acción permitirá restablecer una mejor conexión para el cliente y así evitar la saturación que se presentaba.



- Segundo caso: Si el resultado indica que la generación del módem del clienteActual es incompatible con la antena a la que está conectado. Basándose en esta información, se ofrecerá una recomendación de una nueva antena que sea accesible y compatible para que el usuario pueda decidir si realizar el cambio o no.

Para obtener la nueva antena recomendada, se utiliza el método 'rastrearGeneracionCompatible' de la clase 'Antena'. Para acceder a este método, se utiliza la antena que está asociada al módem del 'clienteActual'. Este método recibe como argumentos el arreglo de 'antenasSede' que se instanció al inicio de la funcionalidad, así como el módem (objeto) asociado al 'clienteActual'.

```

1120 elif "incompatible" in resultado:
1121
1122     messagebox.showinfo(title = "Incompatibilidad",
1123         message = resultado,
1124         detail = "El test ha detectado problemas con tu red.\nLa generación de tu router es incompatible con la antena a la que estás conectado.")
1125     self.frame_display.config(borderwidth=2, relief="solid", height=200, width=500)
1126     text_antena = tk.Label(self.frame_display, bg="grey85", text="Se ha encontrado una antena \ncompatible y accesible:")
1127     text_antena.grid(row=0, column=0, padx=50, pady=10, sticky="w")
1128     text_antenas = tk.Label(self.frame_display, bg="grey85", text=antena.rastrearGeneracionCompatible(antenas_sede, self.cliente._getModem()))
1129     text_antenas.grid(row=1, column=0, padx=50, pady=10, sticky="w")
1130
1131     text_pregunta = tk.Label(self.frame_display, bg="grey85", text="¿Cambiar?")
1132
1133     text_pregunta.grid(row=2, column=0, padx=50, sticky="w")
1134
1135     si_button = tk.Button(self.frame_display, text=" Sí ", command=function)
1136     si_button.grid(row=2, column=1, padx=10)
1137
1138     no_button = tk.Button(self.frame_display, text=" No ", command=function2)
1139     no_button.grid(row=2, column=2)

```

Ubicación: uiMain/ventanaUsuario, línea 1120.

Una vez realizados los cálculos pertinentes y se haya encontrado una nueva antena que cumpla con las características requeridas, el método devolverá ese objeto tipo 'Antena' para hacer la respectiva recomendación al cliente.

Se notificará y explicará al usuario sobre el problema encontrado en su red a través de una ventana emergente. Una vez se haga la recomendación, se le pregunta al usuario si desea hacer el respectivo cambio.

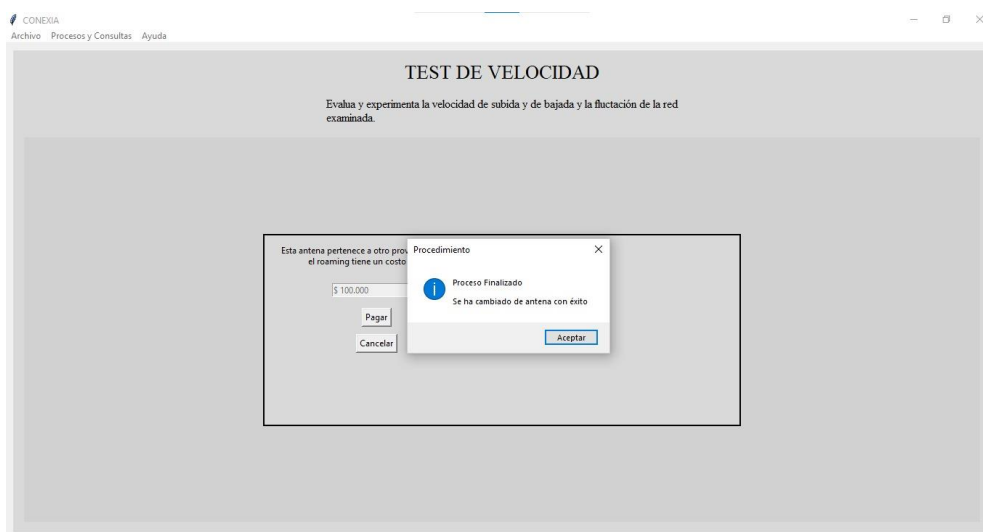


Si la respuesta es “Sí”, se ejecuta el siguiente código (en el primer caso, se creó una instancia de la variable 'antena_nueva' de tipo 'Antena' para almacenar la recomendada al usuario y esta antena se obtiene de la misma manera):

```
1066     if antena_nueva.getProveedor().getNombre()==self.cliente_.getProveedor().getNombre():
1067         self.cliente_.getModem().setAntenaAsociada(antena_nueva)
1068         messagebox.showinfo(title = "Procedimiento",
1069                             message = "Proceso Finalizado",
1070                             detail = "Se ha cambiado de antena")
```

Ubicación: uiMain/ventanaUsuario, línea 1066.

Después se verifica si el nombre del proveedor asociado a 'antenaNueva' es igual al nombre del proveedor asociado al 'clienteActual'. Si este condicional se cumple, significa que la 'antenaNueva' está asociada al mismo proveedor que el 'clienteActual', por lo tanto, se le asignará esa antena al objeto Router que tiene asociado sin ningún tipo de costo o acción adicional. Después de realizar esta asignación, se le informará al usuario que el proceso ha finalizado con éxito y la ejecución de la funcionalidad se dará por finalizada.



Por el contrario, si el nombre del proveedor asociado es diferente para ambos objetos, se ejecutará el siguiente bloque de código.

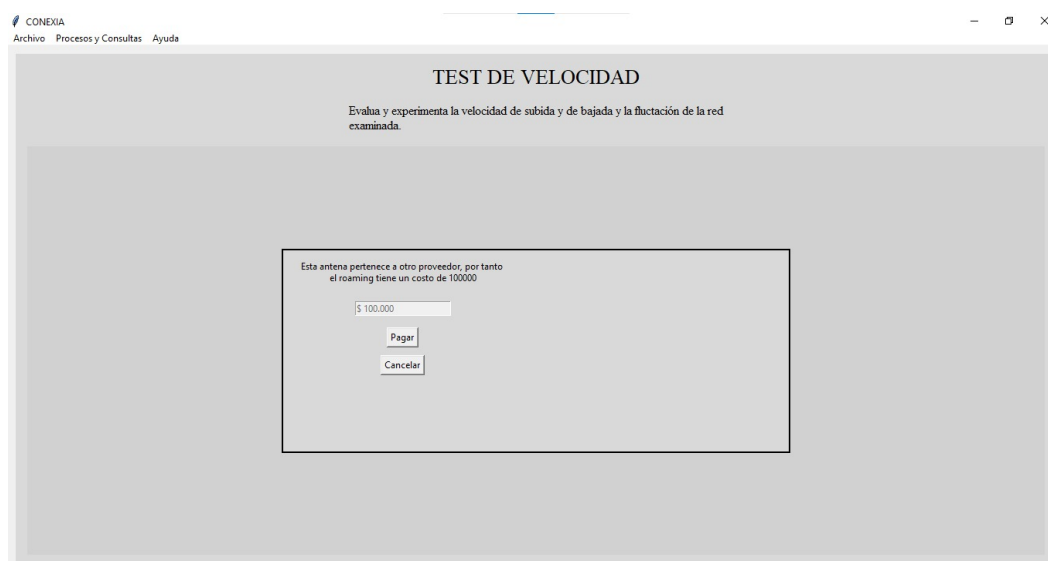
```

1071 else:
1072     self.frame_display.destroy()
1073     self.frame_display=tk.Frame(frameInformacion, bg="grey85", height=200, width=500)
1074     self.frame_display.config(borderwidth=2, relief="solid")
1075     self.frame_display.place(relx=0.5, rely=0.5, relheight=0.5,relwidth=0.5, anchor="center")
1076
1077     text_info=tk.Label(self.frame_display,text="Esta antena pertenece a otro proveedor, por tanto
1078     text_info.config(bg="grey85")
1079     text_info.grid(row=0,column=0,padx=20,pady=10)
1080     entry_costo=tk.Entry(self.frame_display)
1081     entry_costo.insert(tk.END,"$ 100.000")
1082     entry_costo.config(state="disabled")
1083     entry_costo.grid(row=1,column=0,pady=10)
1084     pagar_button=tk.Button(self.frame_display,text="Pagar",command=pagar)
1085     pagar_button.grid(row=2,column=0,pady=5)
1086     cancelar_button=tk.Button(self.frame_display,text="Cancelar",command=cancelar)
1087     cancelar_button.grid(row=3,column=0,pady=5)

```

Ubicación: uiMain/ventanaUsuario, línea 1071.

Aquí se define una entrada llamada 'costo' que representa el precio estándar a cobrar a cada usuario en caso de que su proveedor asociado sea diferente al proveedor de la 'antenaNueva' y se informa al usuario por consola sobre el precio a pagar y, al hacerlo, el proceso se completa con éxito, finalizando así la funcionalidad.



- Tercer caso: Si el resultado indica que el clienteActual está fuera de la zona de cobertura de su

antena asociada, es decir, que es inaccesible debido a problemas de cobertura. Se le informará al cliente y mostrará por pantalla el problema detectado con una breve descripción de la situación. Luego, se le sugiere al usuario cambiar a una antenna nueva que sea accesible y compatible, utilizando el método 'rastrearGeneracionCompatible' de la clase 'Antena'. Este método se invoca con el objeto Antena asociado al router del cliente, recibe como parámetros el arreglo de 'antenas_Sede' y el módem asociado al 'clienteActual'. De esta manera, se obtiene una antenna recomendada que cumpla con las características necesarias.

```

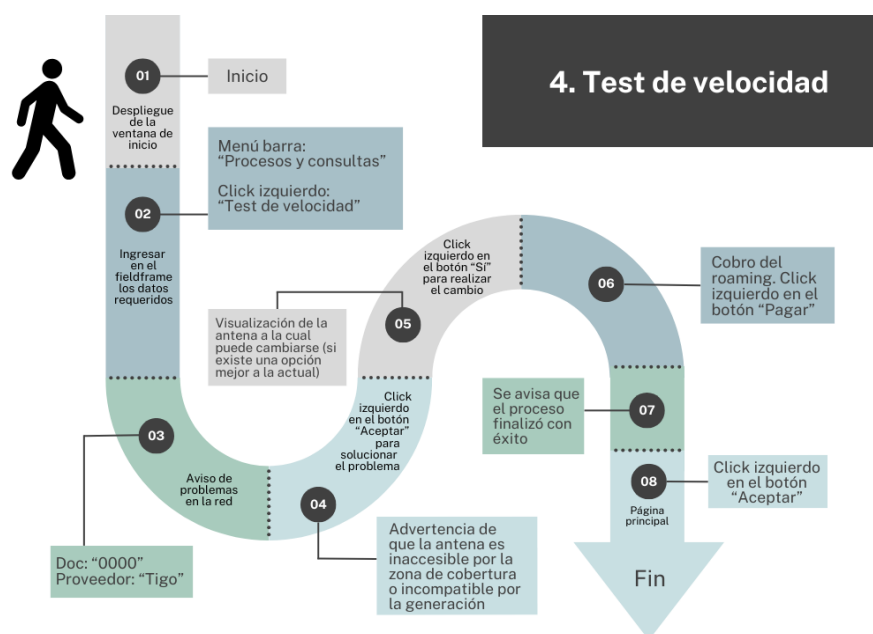
1141 elif "Inaccesible" in resultado:
1142
1143     messagebox.showinfo(title = "Inaccesibilidad",
1144     message = resultado,
1145     detail = "El test ha detectado problemas con tu red.\n Te encuentras fuera de la zona de cobertura de la antenna que tienes asociada.")
1146     self.frame_display.config(borderwidth=2, relief="solid",height=200, width=500)
1147     text_antena = tk.Label(self.frame_display, bg="grey85", text="Se ha encontrado una antenna \ncompatible y accesible:")
1148     text_antena.grid(row=0, column=0, padx=50, pady=10,sticky="w")
1149     text_antenas = tk.Label(self.frame_display, bg="grey85", text=antena.rastrearGeneracionCompatible(antenas_sede,self.cliente_.getModem()))
1150     text_antenas.grid(row=1, column=0, padx=50, pady=10,sticky="w")
1151
1152
1153     text_pregunta=tk.Label(self.frame_display,bg="grey85",text="¿ Deseas cambiarte ?")
1154
1155     text_pregunta.grid(row=2,column=0,padx=50, sticky="w")
1156
1157     si_button = tk.Button(self.frame_display, text="Si", command=function)
1158     si_button.grid(row=2,column=1, padx=10)
1159
1160     no_button = tk.Button(self.frame_display, text="No", command=function2)
1161     no_button.grid(row=2,column=2)

```

Ubicación: uiMain/ventanaUsuario, línea 1141.

A partir de ese punto, el resto de la funcionalidad es similar al segundo condicional ('Incompatible'). Por lo tanto, se pregunta al usuario si desea cambiar a la antenna recomendada. En caso de que la respuesta sea afirmativa, se verifica si el proveedor de la antenna nueva es igual al proveedor asociado al 'clienteActual'. Si son iguales, se establece la 'antenaNueva' como la antenna actual del cliente. De lo contrario, se cobra un precio estándar que el usuario debe ingresar a través de la consola. Una vez se verifique que el pago ha sido válido, se establece la 'antenaNueva' como la antenna actual y se da por finalizada la funcionalidad.

A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad, mediante un caso de prueba.



Nota: en caso de que el resultado indique saturación, aparecerá un cuadro en donde se debe seleccionar la antenna se desea eliminar y se continúa con el proceso normalmente.

5.5. Funcionalidad 5: Reporte de fallas en el servidor.

En el siguiente bloque de código se ejecuta otro método estático importante de la funcionalidad llamado 'buscarCliente' de la clase 'Cliente'. Este método recibe como parámetros: la sede (String) donde se encuentra el servidor ('servidorLocalidad'), el nombre (String) del plan del cual se quiere obtener los clientes y el objeto 'ProveedorInternet' al cual está asociado el servidor. Al ingresar estos argumentos, el método devolverá la lista de clientes correspondiente al plan deseado en la localidad seleccionada, del proveedor asociado. Para completar la funcionalidad, se invoca este método tres veces para obtener el arreglo de clientes de cada plan.

```
1242 # //SE IMPLEMENTA TRES VECES EL SEGUNDO METODO DE LA FUNCIONALIDAD--SE SEPARAN LOS CLIENTES DEL PROVEEDOR DE ACUERDO CON SU PLAN
1243 clientesBasic = Cliente.buscarCliente1(servidorLocalidad.getSede(), "BASIC",servidorLocalidad.getProveedor())
1244 clientesStandard = Cliente.buscarCliente1(servidorLocalidad.getSede(), "STANDARD",servidorLocalidad.getProveedor())
1245 clientesPremium = Cliente.buscarCliente1(servidorLocalidad.getSede(), "PREMIUM",servidorLocalidad.getProveedor())
```

Ubicación: uiMain/ventanaUsuario, línea 1242.

Teniendo en cuenta la información obtenida, se procede a ejecutar el siguiente bloque de código, donde se implementa otro método relevante de la funcionalidad llamado 'intensidadFlujoClientes' de la clase 'Router'. Este método se invoca a través de un objeto Router aleatorio generado por el método estático 'adminRouter'.

Una vez instanciado el objeto 'Router', se pasa como parámetros uno de los arreglos de clientes por plan generados anteriormente, la variable 'servidorLocalidad' y un booleano (true o false) que determinará si se calculan las intensidades de flujo de red reales (true) o las estimadas (false). El método 'intensidadFlujoClientes' realiza una serie de cálculos en función de los parámetros ingresados y devuelve una lista de enteros que representa las intensidades de flujo de red de los clientes por plan.

Para continuar con la funcionalidad, se necesitan los datos de las intensidades de flujo de red, tanto reales como estimadas, de los clientes presentes en las listas generadas anteriormente y para almacenar esta información, se crean seis variables que guardarán los datos de las intensidades de flujo de red correspondientes a cada cliente y plan, respectivamente.

```
1247 # SE IMPLEMENTA EL TERCER METODO DE LA FUNCIONALIDAD
1248
1249 # SE CALCULA LAS INTENSIDADES REALES DE LOS CLIENTES, PERO ESTO ES CLASIFICADO POR PLAN
1250 intensidadBasic = Router.adminRouter().intensidadFlujoClientes(clientesBasic,servidorLocalidad, True)
1251 intensidadStandard = Router.adminRouter().intensidadFlujoClientes(clientesStandard, servidorLocalidad, True)
1252 intensidadPremium = Router.adminRouter().intensidadFlujoClientes(clientesPremium,servidorLocalidad, True)
1253
1254 # SE CALCULA LAS INTENSIDADES QUE DEBERIAN LLEGARLE A LOS CLIENTES, PERO ESTO ES CLASIFICADO POR PLAN
1255 intensidadB = Router.adminRouter().intensidadFlujoClientes(clientesBasic,servidorLocalidad, False)
1256 intensidadS = Router.adminRouter().intensidadFlujoClientes(clientesStandard,servidorLocalidad, False)
1257 intensidadP = Router.adminRouter().intensidadFlujoClientes(clientesPremium,servidorLocalidad, False)
```

Ubicación: uiMain/ventanaUsuario, línea 1247.

Una vez se obtienen las listas, se calcula el promedio para cada una de los arreglos de enteros creados anteriormente, con el objetivo de hacer las respectivas comparaciones y generar el reporte solicitado por el proveedor. Se ubican en un bloque de try/except para controlar el error causado por una posible división por cero, así como se ve en el siguiente bloque de código:

```

1259     # PROMEDIO INTENSIDADES REALES
1260     try:
1261         PromedioBasic = sum(intensidadBasic)/ len(intensidadBasic)
1262     except ZeroDivisionError:
1263         PromedioBasic = 0
1264
1265     try:
1266         PromedioStandard = sum(intensidadStandard) / len(intensidadStandard)
1267     except ZeroDivisionError:
1268         PromedioStandard = 0
1269
1270     try:
1271         PromedioPremium = sum(intensidadPremium) / len(intensidadPremium)
1272     except ZeroDivisionError:
1273         PromedioPremium = 0
1274
1275     # PROMEDIO INTENSIDADES ADECUADAS
1276     try:
1277         PromedioB = sum(intensidadB) / len(intensidadB)
1278     except ZeroDivisionError:
1279         PromedioB = 0
1280
1281     try:
1282         PromedioS = sum(intensidadS) / len(intensidadS)
1283     except ZeroDivisionError:
1284         PromedioS = 0
1285
1286     try:
1287         PromedioP = sum(intensidadP) / len(intensidadP)
1288     except ZeroDivisionError:
1289         PromedioP = 0

```

Ubicación: uiMain/ventanaUsuario, línea 1259 - 1289.

Una vez obtenidos estos promedios, se verifica si algún promedio de intensidad de flujo real es menor que el promedio estimado con ayuda del siguiente condicional:

```

1291     if ((PromedioBasic < PromedioB) or (PromedioStandard < PromedioS) or (PromedioPremium < PromedioP)):

```

Ubicación: uiMain/ventanaUsuario, línea 1291.

Si se encuentra algún promedio real más bajo que el estimado, indica la presencia de problemas en el servidor, y se procede a brindar recomendaciones específicas. Caso tal de que si haya un promedio real menor a un promedio estimado, se ejecuta el siguiente bloque de código:

```

1299     #PROMEDIO DE LAS DISTANCIAS OPTIMAS
1300     try:
1301
1302         PromedioDB = round(sum(dOptBasic) / len(dOptBasic) , 2)
1303     except ZeroDivisionError:
1304         PromedioDB = 0
1305
1306     try:
1307         PromedioDS = round(sum(dOptStandard) / len(dOptStandard), 2)
1308     except ZeroDivisionError:
1309         PromedioDS = 0
1310
1311     try:
1312         PromedioDP = round(sum(dOptPremium) / len(dOptPremium), 2)
1313     except ZeroDivisionError:
1314         PromedioDP = 0

```

Ubicación: uiMain/ventanaUsuario, línea 1299.

Aquí se utiliza otro método importante de la funcionalidad, llamado 'distanciasOptimas', de la

clase 'Servidor'. Este método se llama a través del apuntador 'servidorLocalidad' y recibe como parámetros una de las listas de clientes por plan y su respectiva lista de intensidades de flujo estimadas. El método realiza cálculos específicos con la información de estas listas y devuelve una lista de enteros que representan las distancias óptimas que cada cliente debería tener, con respecto al servidor, para lograr una mejor intensidad de flujo de red.

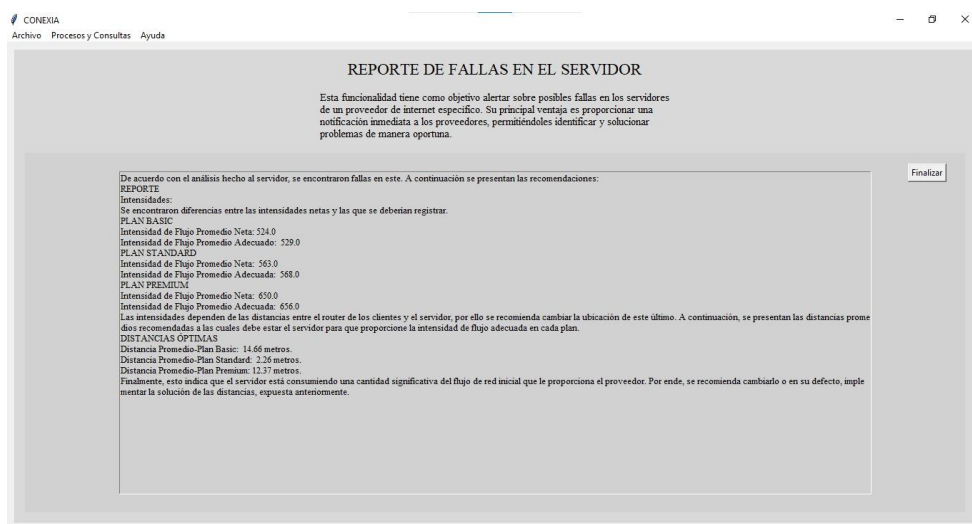
```

1293 # SE CALCULAN LAS DISTANCIAS PROMEDIOS A LAS QUE DEBERIA ENCONTRARSE EL SERVIDOR DE SUS CLIENTES
1294 # DISTANCIAS OPTIMAS POR PLAN
1295 dOptBasic = servidorLocalidad.distanciasOptimas(clientesBasic, intensidadB)
1296 dOptStandard = servidorLocalidad.distanciasOptimas(clientesStandard, intensidadS)
1297 dOptPremium = servidorLocalidad.distanciasOptimas(clientesPremium, intensidadP)

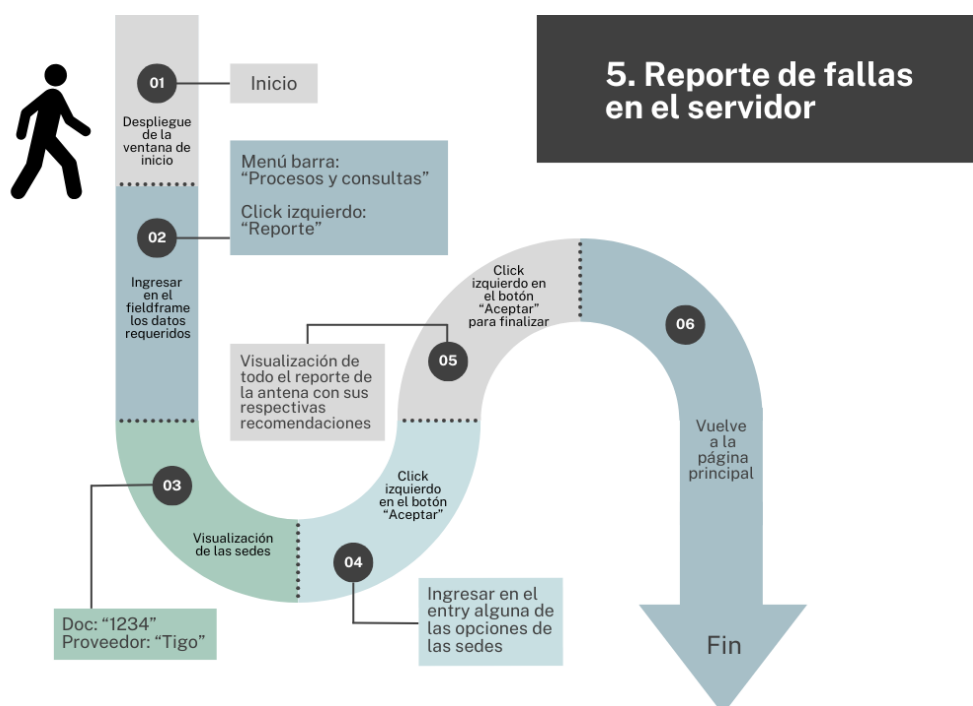
```

Ubicación: uiMain/ventanaUsuario, línea 1293.

Un ejemplo de lo que saldría en pantalla una vez se haya encontrado la información anteriormente mencionada es:



A continuación, el siguiente diagrama de interacción ilustra cómo se establece la relación entre el usuario y el código en esta funcionalidad, mediante un caso de prueba.



6. Manejo de excepciones: En Python, las excepciones permiten manejar errores de forma controlada usando try-except y raise, lo que asegura que el programa siga funcionando correctamente ante errores inesperados.

En este proyecto, se implementaron excepciones personalizadas, organizadas en la carpeta excepciones, que contiene tres clases principales: ErrorAplicacion, ExceptionC1 y ExceptionC2. Estas clases sirven como base para otras excepciones personalizadas, facilitando un manejo de errores más estructurado en el código. A continuación, se describirá cada una de estas clases.

6.1. ErrorAplicacion: Esta clase es la excepción base, se hereda de la clase Exception de Python y se usa para manejar errores generales dentro de la aplicación. La clase toma un mensaje como argumento y lo formatea para generar un mensaje de error con el prefijo "Manejo de errores de la Aplicación".

```
1 class ErrorAplicacion(Exception):
2
3     def __init__(self, mensajeEApp):
4         self.mensajeErrorApp = "Manejo de errores de la Aplicación: \n" + mensajeEApp
5         super().__init__(self.mensajeErrorApp)
```

Ubicación: uiMain/excepciones/errorAplicacion.py, línea 1.

Al utilizar esta clase, cualquier error relacionado con el manejo general de la aplicación puede ser capturado y mostrado de manera uniforme, indicando que se trata de un error relacionado con el funcionamiento interno de la aplicación.

6.2. Exception1 (hereda de ErrorAplicacion): Esta clase proporciona un tipo específico de excepción con un mensaje personalizado. El mensaje se formatea con el prefijo "Exception1".

```
3 class Exception1(ErrorAplicacion):
4
5     def __init__(self, mensajeEx1):
6         self.mensajeEx1 = f"Exception1: {mensajeEx1}"
7         super().__init__(self.mensajeEx1)
```

Ubicación: uiMain/excepciones/exceptionC1.py, línea 3.

Exception1 sirve como una clase base para manejar errores más específicos en la aplicación. Existen 3 clases específicas que heredan de Exception1 y se enfocan en tipos concretos de errores: coordenadas incorrectas, pagos incompletos, datos incorrectos.

1. ErrorCoordenadas: Esta clase maneja los errores cuando las coordenadas proporcionadas por el usuario están fuera del rango esperado, no son válidas o están mal formateadas. El mensaje de error indicará que las coordenadas ingresadas no son correctas y debe corregirse el valor.

```
9 class ErrorCoordenadas(Exception1):
10
11     def __init__(self):
12         self.mensajeEC = f"Las coordenadas están fueran del rango. Vuelve a ingresarlas."
13         super().__init__(self.mensajeEC)
```

Ubicación: uiMain/excepciones/exceptionC1.py, línea 9.

Se garantiza que los datos de ubicación ingresados sean válidos y dentro de los límites aceptables. Se activa cuando las coordenadas no cumplen con los requisitos establecidos en el sistema, como cuando se exceden los rangos permitidos. Por ejemplo:

```

153         if sede=="A":
154             if not(coordenadas[0][0] <= coordenadaX <= coordenadas[0][1] or not(coordenadas[0][2] <= coordenadaY <= coordenadas[0][3]):
155                 raise ErrorCoordenadas()
156
157         elif sede=="B":
158             if not(coordenadas[1][0] <= coordenadaX <= coordenadas[1][1] or not(coordenadas[1][2] <= coordenadaY <= coordenadas[1][3]):
159                 raise ErrorCoordenadas()
160
161         elif sede=="C":
162             if not(coordenadas[2][0] <= coordenadaX <= coordenadas[2][1] or not(coordenadas[2][2] <= coordenadaY <= coordenadas[2][3]):
163                 raise ErrorCoordenadas()
164
165         elif sede=="D":
166             if not(coordenadas[3][0] <= coordenadaX <= coordenadas[3][1] or not(coordenadas[3][2] <= coordenadaY <= coordenadas[3][3]):
167                 raise ErrorCoordenadas()

```

Ubicación: uiMain/ventanaUsuario.py, línea 153.

2. ErrorPagoTotal: Esta clase maneja situaciones donde el pago total ingresado por el usuario no es el esperado. Si el monto pagado no es correcto (por ejemplo, es menor que el total debido), esta excepción se dispara. El mensaje especifica que el valor ingresado no es correcto y solicita que se ingrese la cantidad completa.

```

15 class ErrorPagoTotal(Exception1):
16
17     def __init__(self):
18         self.mensajeEP = f"El valor ingresado no es correcto. Ingrese la cantidad completa."
19         super().__init__(self.mensajeEP)

```

Ubicación: uiMain/excepciones/exceptionC1.py, línea 15.

Asegura que los pagos se procesen correctamente. Se activa cuando el pago total ingresado no es suficiente, ayudando a evitar que el sistema acepte pagos incompletos o incorrectos. Por ejemplo:

```

121 except ErrorPagoTotal as e:
122
123     for widget in frameInformacion.wininfo_children():
124         widget.destroy()
125
126     messagebox.showerror(title="Error", message="ERROR PAGO TOTAL", detail=f"{str(e)}")
127     return adquisicionPlan()

```

Ubicación: uiMain/ventanaUsuario.py, línea 121.

3. DatosIncorrectos: Esta clase es utilizada cuando los datos ingresados por el usuario son incorrectos o no cumplen con el formato esperado. Si los datos no coinciden con los requisitos de validación del sistema (por ejemplo, un campo vacío o un valor no válido), se lanza esta excepción. El mensaje de error señala que los datos ingresados no son correctos y se debe verificar la entrada.

```

21 class DatosIncorrectos(Exception1):
22
23     def __init__(self):
24         self.mensajeED = f"Los datos ingresados no son correctos. Verifica y vuelve a digitarlos."
25         super().__init__(self.mensajeED)

```

Ubicación: uiMain/excepciones/exceptionC1.py, línea 21.

Mejora la calidad de los datos ingresados por el usuario. Esta excepción se activa cuando los valores proporcionados no cumplen con las restricciones necesarias, lo que ayuda a asegurar que solo se procese información válida y completa. Por ejemplo:

```

884         if proveedorActual==None:
885             raise DatosIncorrectos()
886
887         clienteActual = proveedorActual.verificarCliente(documento, )
888
889         if clienteActual == None:
890             raise DatosIncorrectos()

```

Ubicación: uiMain/ventanaUsuario.py, línea 884.

6.3. Exception2 (hereda de ErrorAplicacion): Similar a Exception1, esta clase también hereda de ErrorAplicacion pero está orientada a manejar otro tipo de excepciones con un formato diferente. El mensaje de error incluye el prefijo "Exception2".

```

3 class Exception2(ErrorAplicacion):
4
5     def __init__(self, mensajeEx2):
6         self.mensajeEx2= f"Exception2. {mensajeEx2}"
7         super().__init__(self.mensajeEx2)

```

Ubicación: uiMain/excepciones/exceptionC2.py, línea 3.

Exception2 sirve para gestionar otro tipo de excepciones, especialmente aquellas que están relacionadas con la entrada del usuario, como elecciones incorrectas o datos mal formateados. Existen 3 clases específicas que heredan de Exception2.

1. ErrorOpElegida: Esta clase maneja los casos en los que el usuario selecciona una opción no válida de un conjunto predefinido. Si el usuario ingresa una opción que no está dentro de las opciones permitidas, esta excepción se dispara. El mensaje de error informará que la opción elegida no es válida y sugerirá ingresar una nueva opción.

```

9 class ErrorOpElegida(Exception2):
10
11     def __init__(self):
12         self.mensajeEO = f"La opción que elegiste no es válida. Ingresa de nuevo."
13         super().__init__(self.mensajeEO)

```

Ubicación: uiMain/excepciones/exceptionC2.py, línea 9.

Asegura que el usuario seleccione solo opciones válidas dentro del sistema. Esta excepción se activa cuando se hace una selección incorrecta, lo que ayuda a guiar al usuario para que elija opciones dentro de las alternativas correctas. Por ejemplo:

```

1304 except ErrorOpElegida as e:
1305
1306     for widget in frameInformacion.winfo_children():
1307         widget.destroy()
1308
1309     messagebox.showerror(title="Error", message="ERROR OPCIÓN ELEGIDA", detail=f"{str(e)}")
1310     return reporte()

```

Ubicación: uiMain/ventanaUsuario.py, línea 1304.

2. ErrorLetra: Esta clase se dispara cuando el usuario ingresa una letra en lugar de un número cuando se esperaba un valor numérico. Si el sistema recibe un tipo de dato incorrecto, como una letra en lugar de un número, se lanza esta excepción. El mensaje informa al usuario que debe ingresar un valor numérico correcto.

```

15 class ErrorLetra(Exception2):
16
17     def __init__(self):
18         self.mensajeEL = f"Ingresaste una letra en lugar de un número. Ingresa un valor correcto."
19         super().__init__(self.mensajeEL)

```

Ubicación: uiMain/excepciones/exceptionC2.py, línea 15.

Garantiza que los datos ingresados sean numéricos cuando se esperan valores de tipo número. Se activa cuando el usuario comete un error de tipo de dato, mejorando la robustez del sistema. Por ejemplo:

```

904 except ErrorLetra as e:
905     messagebox.showerror(title="Error", message="ERROR TIPO DATO", detail=f"{str(e)}")
906     for widget in frameInformacion.winfo_children():
907         widget.destroy()
908     return mejoraTuPlan()

```

Ubicación: uiMain/ventanaUsuario.py, línea 904.

3. ErrorEspacios: Esta clase maneja los casos donde el usuario deja espacios en blanco en los campos obligatorios del formulario. Si algún campo obligatorio está vacío o tiene espacios en blanco, esta excepción es lanzada. El mensaje de error informa que deben ingresar todos los datos sin dejar espacios vacíos.

```
21 class ErrorEspacios(Exception2):
22
23     def __init__(self):
24         self.mensajeES= f"Estas dejando espacios en blanco. Ingresar todos los datos solicitados."
25         super().__init__(self.mensajeES)
```

Ubicación: uiMain/excepciones/exceptionC2.py, línea 21.

Mejora la completitud de los datos ingresados, asegurando que el usuario no omita ningún campo importante. Se activa cuando el sistema detecta que el usuario ha dejado espacios vacíos en los campos requeridos, lo que garantiza que todos los datos necesarios estén completos. Por ejemplo:

```
95         if pago == "":
96             raise ErrorEspacios()
```

Ubicación: uiMain/ventanaUsuario.py, línea 95.

7. FieldFrame: es un contenedor en la interfaz gráfica (usualmente con Tkinter) que agrupa y organiza elementos como campos de entrada, botones y etiquetas, facilitando la creación de formularios y una interfaz más ordenada.

7.1. Descripción.

Para esta clase se describe su implementación como que, se extiende `tk.Frame` y se encarga de generar un conjunto de etiquetas (`Label`) y campos de entrada (`Entry`) de manera estructurada.

Su objetivo principal es simplificar la captura de datos en la interfaz gráfica de la aplicación. Esta clase permite definir criterios de entrada, asignar valores iniciales y controlar el estado de los campos.

Los componentes de estas clases son:

1. Constructor (`__init__`):

- Recibe los parámetros `parent`, `tituloCriterios`, `criterios`, `tituloValores`, `valores` y `habilitado`.
- Inicializa los atributos `criterios`, `valores`, `habilitado`, `labels` y `entries`.

```

11  # CONSTRUCTOR
12  def __init__(self, parent, tituloCriterios, criterios, tituloValores, valores=None, habilitado=None):
13
14      super().__init__(parent)
15      self.criterios = criterios
16
17      if valores is not None:
18          self.valores = valores
19      else:
20          self.valores = [""] * len(criterios)
21
22      if habilitado is None:
23          self.habilitado = [False] + [True] * (len(criterios) - 1)
24
25      self.labels = []
26      self.entries = []

```

Ubicación: `uiMain/fieldFrame.py`, línea 11.

- Crea y posiciona etiquetas y campos de entrada en la interfaz.

```

28  label_criterios = tk.Label(self, text=tituloCriterios, font=("Arial", 12, "bold"), bg="lightgray", fg="#4B0082")
29  label_valores = tk.Label(self, text=tituloValores, font=("Arial", 12, "bold"), bg="lightgray", fg="#4B0082")
30
31  label_criterios.grid(row=0, column=0, sticky='we', padx=10, pady=10)
32  label_valores.grid(row=0, column=1, sticky='we', padx=10, pady=10)

```

Ubicación: `uiMain/fieldFrame.py`, línea 28.

- Controla la habilitación de los campos y su ubicación en el frame.

```

34  for i, criterio in enumerate(criterios, start=1):
35
36      label = tk.Label(self, text=criterio)
37      entry = tk.Entry(self, state="normal")
38      entry.insert('end', self.valores[i-1])
39
40      if i == 1: # Primer campo
41          estado = "disabled"
42      else: # Otros campos
43          estado = "normal"
44
45      entry.config(state=estado)
46
47      label.grid(row=i, column=0, sticky='we', padx=30, pady=10)
48      entry.grid(row=i, column=1, sticky='we', padx=30, pady=10)
49      self.labels.append(label)
50      self.entries.append(entry)
51

```

Ubicación: `uiMain/fieldFrame.py`, línea 34.

2. Método `getValor(criterio)`:

- Obtiene el valor ingresado en un campo según el criterio especificado.
- Valida que el campo no esté vacío; en caso contrario, lanza la excepción `ErrorEspacios`.

```

55     # Obtener el valor de cada campo
56     def getValor(self, criterio):
57         index = self.criterios.index(criterio)
58
59         if (self.entries[index].get()) == "":
60             raise ErrorEspacios()
61         else:
62             return self.entries[index].get()

```

Ubicación: `uiMain/fieldFrame.py`, línea 55.

3. Método `borrarValores()`:

- Recorre todos los Entry y elimina su contenido.

```

64     # Eliminar el valor digitado en los campos
65     def borrarValores(self):
66         for entry in self.entries:
67             entry.delete(0,tk.END)

```

Ubicación: `uiMain/fieldFrame.py`, línea 64.

Además, el alcance y la funcionalidad en la Interfaz Gráfica: La clase `FieldFrame` es utilizada en varias funcionalidades de la aplicación para capturar datos de usuario de manera organizada. Su implementación facilita la reutilización de código y mantiene la coherencia visual en los formularios de entrada. Se utiliza dentro de contenedores (Frame) específicos, junto con botones de acción como "Aceptar" y "Borrar".

7.2. Ejemplos de uso en la Interfaz.

Para la funcionalidad 1: Adquisición de un Plan:

```

273     # SE CONSTRUYE EL FIELDFRAME QUE SOLICITA LOS DATOS CORRESPONDIENTES
274     criterios = ["Tipo de Usuario", "Coordenada en X", "Coordenada en Y"]
275     fp =FieldFrame(frameInformacion, "Datos solicitados", criterios, "Valor", ["Cliente","", ""])
276     fp.configure(borderwidth=1, relief="groove",bg="gray82")
277     fp.place(relx=0.15, rely=0.45, relheight=0.5)

```

Ubicación: `uiMain/ventanaUsuario.py`, línea 273.

Para la funcionalidad 5: Test de velocidad:

```

979 self.titulo.config(text="TEST DE VELOCIDAD", font=("Times", 20))
980 self.descripcion.config(wraplength=500, text =
981     "Evalua y experimenta la velocidad de subida y de bajada y la fluctación de la red examinada.",
982     font=("Times",12))
983
984 criterios = ["Tipo de Usuario","ID", "Proveedor"]
985 valores=["\tCliente","", ""]
986
987 self.frame_display=tk.Frame(frameInformacion, bg="gray95", height=100, width=500)
988 self.frame_display.pack( expand=True, pady=20, anchor="n")
989
990 fp = FieldFrame(self.frame_display, "Datos", criterios, "Valor", valores)
991 fp.configure(borderwidth=0,bg="gray95",)
992 fp.pack(fill="y",pady=3, padx=5)

```

Ubicación: uiMain/ventanaUsuario.py, línea 979.

En estas funcionalidades, FieldFrame estructura la entrada de datos y está integrado con botones para validar y limpiar los datos ingresados.

Viéndose cada interfaz para solicitar datos de esta manera, según corresponda el caso:

The screenshot shows a window titled 'CONEXIA' with a menu bar containing 'Archivo', 'Procesos y Consultas', and 'Ayuda'. The main content area has a title 'ADQUISICIÓN DE UN PLAN' and a descriptive paragraph: 'Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.' Below this is a form with two columns: 'Datos solicitados' and 'Valor'. The form contains four rows of input fields: 'Tipo de Usuario' (with 'Cliente' selected), 'Nombre', 'Documento', and 'Sede'. At the bottom of the form are two buttons: 'Aceptar' and 'Borrar'.

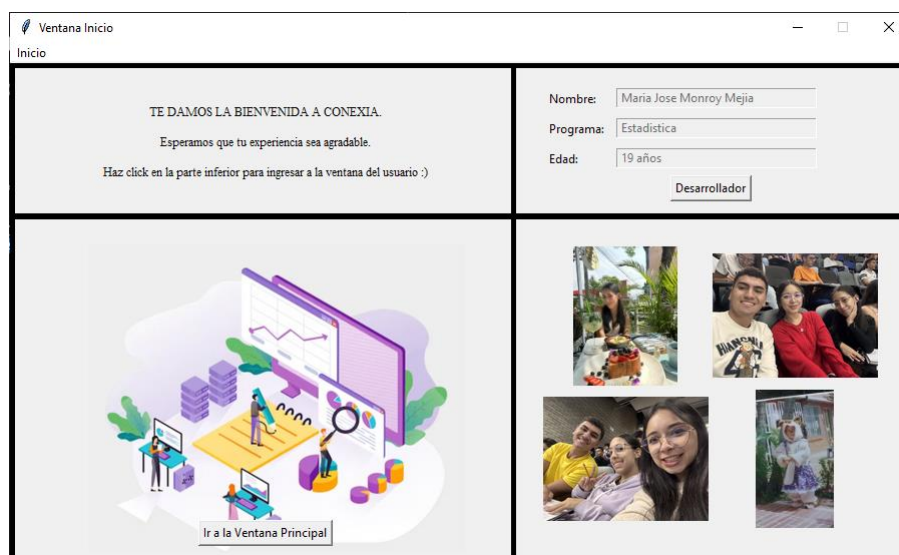
Datos solicitados	Valor
Tipo de Usuario	Cliente
Nombre	
Documento	
Sede	

Aceptar Borrar

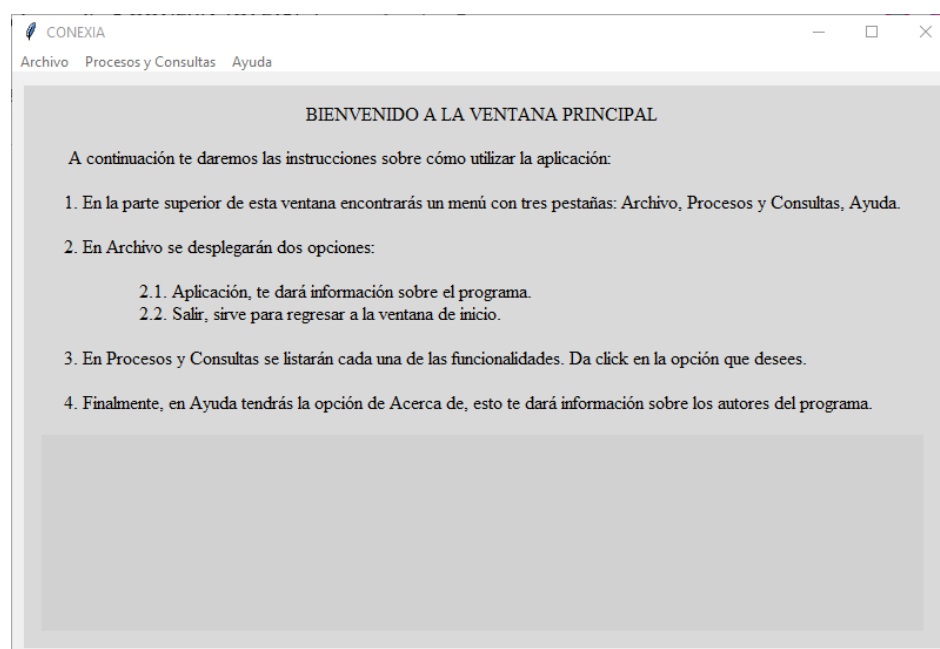
8. Manual de usuario.

Bienvenido al sistema de gestión de proveedores de internet. Este sistema tiene como objetivo facilitar la administración de clientes, planes de servicio, y otros aspectos relacionados a la cobertura y el rendimiento de la red.

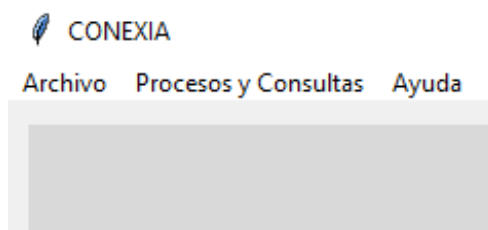
En este manual, aprenderás cómo interactuar con el sistema a través de las funcionalidades disponibles.



Esta es la ventana de inicio del programa, donde te damos una bienvenida, al lado derecho arriba, podrás desplazarte dando click al botón desarrollador por todas las personas que hacen de este programa una realidad, en la parte inferior izquierda, veras un boton “ir a la ventana principal” y este te llevara a la siguiente ventana:



Donde se explican las instrucciones para navegar en la aplicación. Y en la barra superior están las opciones que pueden ser elegidas.



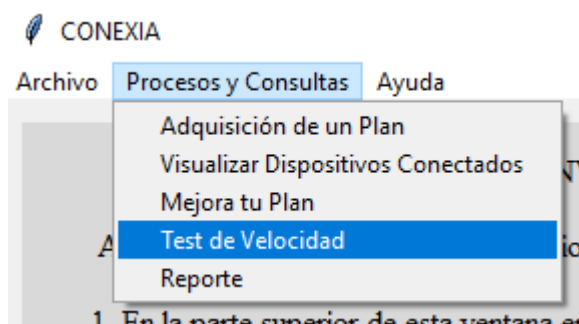
En la parte superior izquierda encontrarás “Archivo”, “Procesos y consultas” y “Ayuda”

En la opción de “archivo” se desplegará la opción de Aplicación y salir

Aplicación: Abre una una pestaña informativa sobre la aplicación.

Salir: para volver a la ventana de Inicio.

En la opción “Procesos y consultas” Podrás elegir una de las siguientes opciones;



8.1. “Adquisición de un Plan”: Primero debes ingresar tus datos para adquirir un plan. el botón aceptar te guiará en las siguientes pestañas:

Con el nombre del usuario, su número de cédula, el nombre de la sede en que se encuentre (las opciones son A, B, C o D), por siguiente das aceptar y continuarán los proveedores disponibles de acuerdo a la localidad seleccionada. Estas son las opciones:

- Sede A: los proveedores disponibles son Claro, Tigo, Movistar.
- Sede B: los proveedores disponibles son Claro, Tigo, Wom.
- Sede C: los proveedores disponibles son Tigo, Movistar.

- Sede D: los proveedores disponibles son Tigo, Movistar, Wom.

CONEXIA

Archivo Procesos y Consultas Ayuda

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

Los proveedores disponibles en tu sede son:

1. tigo
2. claro
3. movistar

Digita el número de la opción que deseas:

Aceptar Cancelar

Se ingresa el número correspondiente al proveedor elegido (entre las opciones de su sede) y se mostrarán en pantalla los tipos de planes ofrecidos al público, que siempre se dividen en tres opciones: Basic, Standard y Premium. El precio claramente varía con el tipo de plan elegido y el proveedor que lo ofrece, las opciones disponibles son:

PROVEEDOR	Tipo de plan	Megas de subida	Megas de bajada	Precio (COP)
TIGO	Basic	10	30	\$ 30.000,00
	Standard	30	50	\$ 55.000,00
	Premium	80	100	\$ 87.000,00
CLARO	Basic	15	35	\$ 45.000,00
	Standard	25	45	\$ 76.500,00
	Premium	83	115	\$ 150.000,00
MOVISTAR	Basic	20	30	\$ 50.000,00
	Standard	28	70	\$ 60.000,00
	Premium	70	97	\$ 95.000,00
WOM	Basic	15	25	\$ 25.000,00
	Standard	23	34	\$ 46.000,00
	Premium	50	70	\$ 80.000,00

En pantalla se verá de la siguiente manera:

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

PLAN BASIC:
Megas de subida: 15
Megas de bajada: 35
Precio del plan: 45000

PLAN STANDARD:
Megas de subida: 25
Megas de bajada: 45
Precio del plan: 76500

PLAN PREMIUM:
Megas de subida: 83
Megas de bajada: 115
Precio del plan: 150000

Digita el nombre del plan que deseas:

STANDARD

Aceptar Cancelar proceso

El siguiente paso será ingresar su ubicación (en coordenadas), se especificarán los límites de su zona, sin embargo, los límites en general son:

ZONA	Coordenadas eje X		Coordenadas eje Y	
	Límite inferior	Límite superior	Límite inferior	Límite superior
A	0	40	0	40
B	0	40	60	100
C	50	80	0	30
D	60	100	40	80

Así se ve por consola:

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

Digita las coordenadas de tu ubicación, estas tienen un límite de acuerdo con tu sede.

Límite Inferior en X: 0
Límite Superior en X: 40
Límite Inferior en Y: 0
Límite Superior en Y: 40

Datos solicitados	Valor
Tipo de Usuario	Cliente
Coordenada en X	
Coordenada en Y	

Aceptar Borrar

Luego, se podrá visualizar la factura, donde se incluirá la información de pagos atrasados (en caso de existir) y se deberá cancelar el valor total de la membresía.

ADQUISICIÓN DE UN PLAN

Esta funcionalidad tiene por objetivo que un usuario nuevo pueda registrarse en el sistema y a su vez compre un plan con las características específicas que brinda el proveedor de su preferencia.

Tu plan ha sido configurado correctamente. A continuación puedes visualizar tu factura.

Factura
Usuario: valeria
Cédula: 123
IP: 125.192.135.186
Mes de Activación: FEBRERO
Número de pagos atrasados: 0
Precio total a pagar: 76500

Realiza el pago de tu membresía a continuación (Ingresa el valor a pagar):

Aquí termina esta funcionalidad.

8.2. “Visualizar dispositivos Vinculados”: al seleccionar la opción, se pedirán los siguientes datos en la ventana.

VISUALIZAR DISPOSITIVOS CONECTADOS

Es la encargada de mostrar por pantalla la lista de dispositivos, que se encuentran vinculados a un router asociado a un cliente, presentando por consola algunas de las características básicas que posee cada dispositivo.

Datos	Valor
Tipo de Usuario	Cliente
Documento	1234
Proveedor	tigo
Aceptar	Borrar

Una vez diligenciado estos campos, dale al botón de aceptar, se verificará si no tienes pagos pendientes. Existen dos caminos desde aquí, dependiendo si se cuentan con deudas o no.

- **Si tienes los pagos al día:** Podrás filtrar por tipo de dispositivo, 4g, 5g o todos.



- **Si tienes pagos pendientes:** Esta opción tiene un sistema para verificar los pagos pendientes de las personas, el requisito para pasar el filtro es no tener más de 2 pagos atrasados, por lo que se solicitará un abono a la deuda o pagar la totalidad de la deuda para poder continuar, de lo contrario está la opción de regresar al menú inicial.



Si se escoge la opción 1 de abonar a la deuda se debe abonar el monto necesario para quedar con máximo 2 pagos atrasados, esto nos permitirá continuar con el mismo proceso visto anteriormente.

8.3. “Mejora tu plan”: Al seleccionar la opción, se pedirán los siguientes datos.

CONEXIA
Archivo Procesos y Consultas Ayuda

MEJORA TU PLAN

Es la encargada de verificar si el usuario tiene posibilidad de mejorar su plan. Caso tal de que se pueda y el usuario quiere cambiarse a ese plan sugerido, hara todo el proceso para cambiar el plan anterior por el nuevo.

Datos	Valor
Tipo de Usuario	Cliente
Documento	1789
Nombre	margarita
Proveedor	movistar

Aceptar Borrar

Después de ingresarlos correctamente, se mostrará una recomendación de mejora para el plan del usuario, quien deberá leer cuidadosamente si quiere cambiar su plan actual por el plan que le recomienda el sistema.

CONEXIA
Archivo Procesos y Consultas Ayuda

MEJORA TU PLAN

Es la encargada de verificar si el usuario tiene posibilidad de mejorar su plan. Caso tal de que se pueda y el usuario quiere cambiarse a ese plan sugerido, hara todo el proceso para cambiar el plan anterior por el nuevo.

Al realizar las verificaciones encontramos que este nuevo servicio puede ser mejor para ti.

Te presentamos la siguiente recomendación:

Intensidad de flujo: 517
 Proveedor: tigo
 Plan: STANDARD
 Megas de subida: 30
 Megas de bajada: 50
 Precio: 55000

Tu plan actual tiene las siguientes características:

Intensidad de flujo: 500
 Proveedor: movistar
 Plan: STANDARD
 Megas de subida: 28
 Megas de bajada: 70
 Precio: 60000

¿Deseas cambiar tu plan por la recomendacion dada?

Si No

El usuario puede cambiar el plan si así lo desea, pero sólo si tiene los pagos al día. En caso de no ser así, se le ofrecerá la opción de pagar la totalidad de la deuda.

CONEXIA

Archivo Procesos y Consultas Ayuda

MEJORA TU PLAN

Es la encargada de verificar si el usuario tiene posibilidad de mejorar su plan. Caso tal de que se pueda y el usuario quiere cambiarse a ese plan sugerido, hara todo el proceso para cambiar el plan anterior por el nuevo.

Tienes 4 pagos atrasados, para continuar debes saldar la deuda. Tu factura actualmente está por un valor de \$240000. Ingresar la cantidad:

Aceptar

Al eliminar la deuda por completo, el plan será actualizado con éxito.

CONEXIA

Archivo Procesos y Consultas Ayuda

MEJORA TU PLAN

Es la encargada de verificar si el usuario tiene posibilidad de mejorar su plan. Caso tal de que se pueda y el usuario quiere cambiarse a ese plan sugerido, hara todo el proceso para cambiar el plan anterior por el nuevo.

Fin funcionalidad

 Proceso Finalizado

¡Tu cambio de servidor y proveedor se ha realizado con éxito!

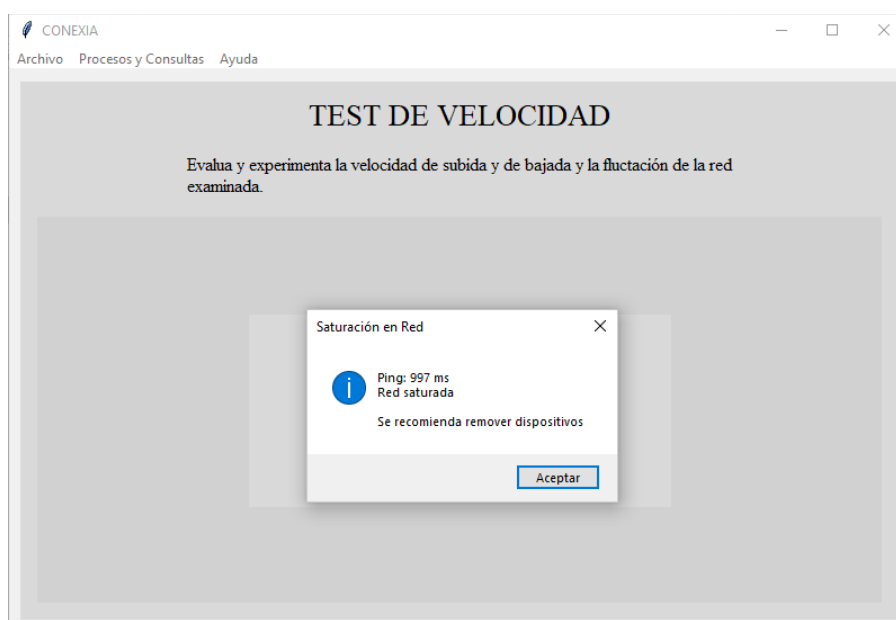
Aceptar

En el caso de no tener deudas, el plan se cambiará automáticamente.

8.4. “Test de velocidad”: Se pedirán los siguientes datos para dar inicio a esta funcionalidad:

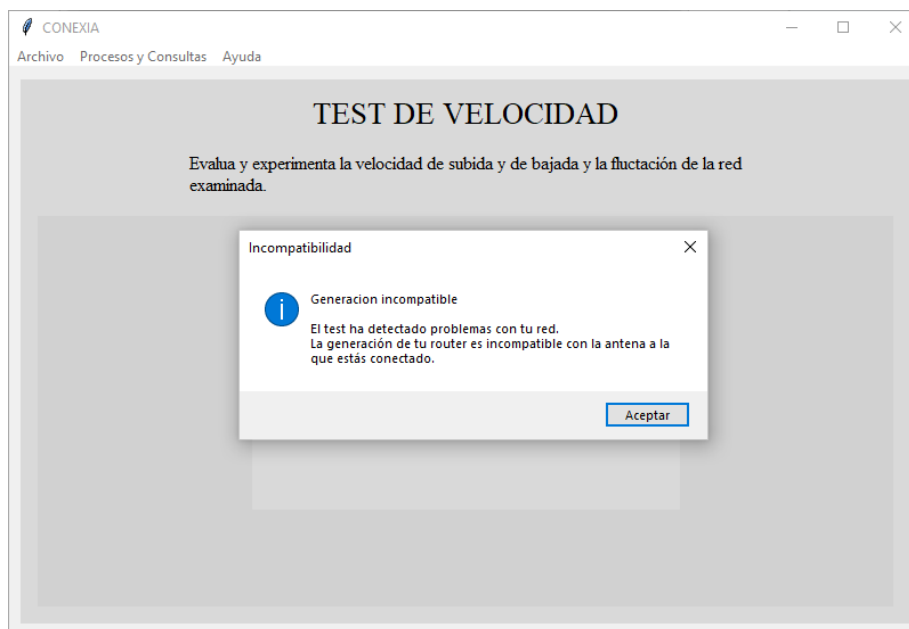
Se hará un test en la red del usuario y se le notificará en caso de encontrar problemas, así mismo, se le ofrecerá una posible solución. Existen 3 tipos de problemas:

- Si la red está saturada, la solución recomendada será remover uno o varios de los dispositivos que hay conectados, para esto ir a la funcionalidad anterior y remover los dispositivos que se desee.

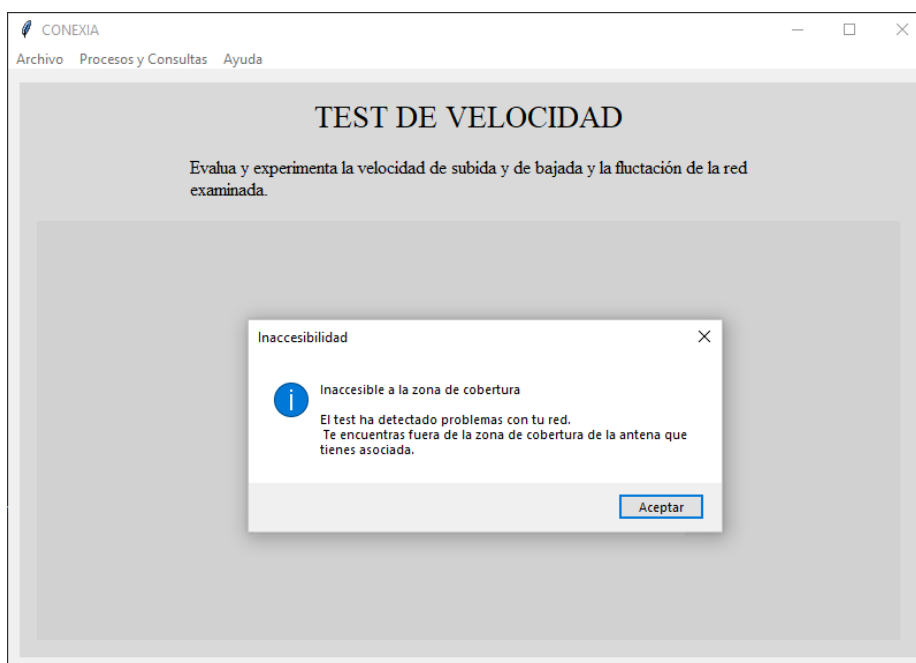


- Si hay incompatibilidad entre el módem del cliente y la antena a la que se conecta, la solución recomendada para este problema es cambiar de antena a una que sea compatible con

la generación.



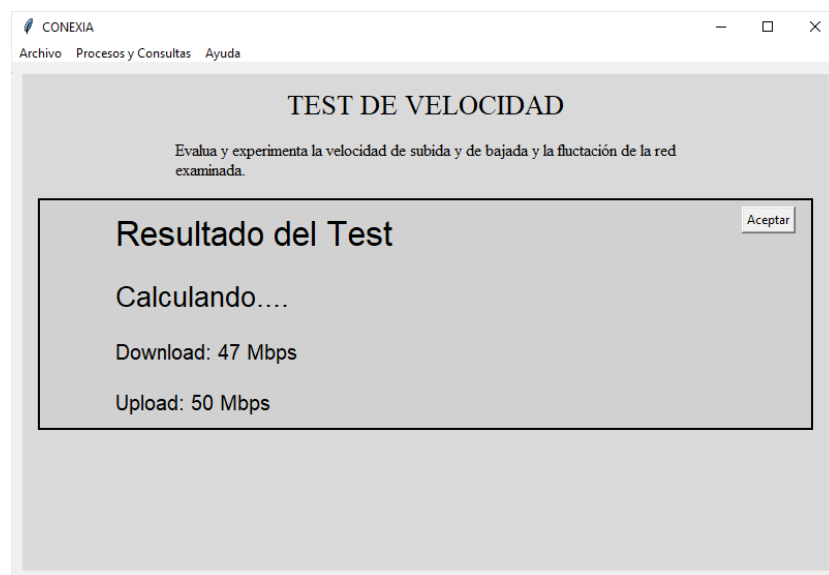
- Si hay inaccesibilidad debido a problemas de cobertura de la antena, la solución recomendada es cambiar la conexión a la antena recomendada para cada caso.



Si presenta este error se le sugerirá cambiar a otra antena si se encuentra una mejor a la actual.



En caso de que el cliente no tenga problemas o que ya los haya solucionado y quiera verificar que sí funcionó, se le mostrará un ping que será aceptable cuando sea menor a 100 ms.



8.5. “Reporte de fallas en el servidor”: Al seleccionar la opción, se le pedirá al usuario ingresar el nombre de su proveedor y, al digitar cualquiera de las opciones (tigo, movistar, claro, wom). Se mostrarán las sedes donde aquel proveedor opera y se nos solicitará escoger de cual sede nos gustaría ver el reporte.

Cada proveedor tiene servicio en diferentes sedes y son:

- Tigo tiene servidores en las sedes A, B, C, D.
- Movistar tiene servidores en las sedes A, C, D.
- Claro tiene servidores en las sedes A, B.
- Wom tiene servidores en las sedes B, D.

Este proceso no tiene mayor complejidad y se repite de manera similar en el resto de sedes, y proveedores, donde las intensidades y distancias cambian para cada uno. Para evitar redundancia en el proceso, sólo se explicará a profundidad la opción para el proveedor Tigo (es el único en las 4 sedes).

CONEXIA

Archivo Procesos y Consultas Ayuda

REPORTE DE FALLAS EN EL SERVIDOR

Esta funcionalidad tiene como objetivo alertar sobre posibles fallas en los servidores de un proveedor de internet específico. Su principal ventaja es proporcionar una notificación inmediata a los proveedores, permitiéndoles identificar y solucionar problemas de manera oportuna.

Datos solicitados	Valor
Tipo de Usuario	Proveedor
Nombre de la compañía	tigo

Aceptar Borrar

Para cada una de las opciones, se revelará la información del rendimiento actual del plan (intensidad de flujo promedio neta) y el rendimiento óptimo (intensidad de flujo promedio adecuada), también se verá una breve explicación de porque puede variar esta intensidad, se mostrarán las distancias recomendadas que debería haber entre el servidor y el cliente, se da una recomendación final y se termina la funcionalidad.

Para la opción 1 (sede A en Tigo):

CONEXIA

Archivo Procesos y Consultas Ayuda

REPORTE DE FALLAS EN EL SERVIDOR

Esta funcionalidad tiene como objetivo alertar sobre posibles fallas en los servidores de un proveedor de internet específico. Su principal ventaja es proporcionar una notificación inmediata a los proveedores, permitiéndoles identificar y solucionar problemas de manera oportuna.

De acuerdo con el análisis hecho al servidor, se encontraron fallas en este. A continuación se presentan las recomendaciones:

REPORTES

Intensidades:

Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.

PLAN BASIC

Intensidad de Flujo Promedio Neta: 512.0

Intensidad de Flujo Promedio Adecuada: 522.0

PLAN STANDARD

Intensidad de Flujo Promedio Neta: 652.5

Intensidad de Flujo Promedio Adecuada: 665.5

PLAN PREMIUM

Intensidad de Flujo Promedio Neta: 508.0

Intensidad de Flujo Promedio Adecuada: 518.0

Las intensidades dependen de las distancias entre el router de los clientes y el servidor, por ello se recomienda cambiar la ubicación de este último. A continuación, se presentan las distancias recomendadas a las cuales debe estar el servidor para que proporcione la intensidad de flujo adecuada en cada plan.

DISTANCIAS ÓPTIMAS

Distancia Promedio-Plan Basic: 11.35 metros.

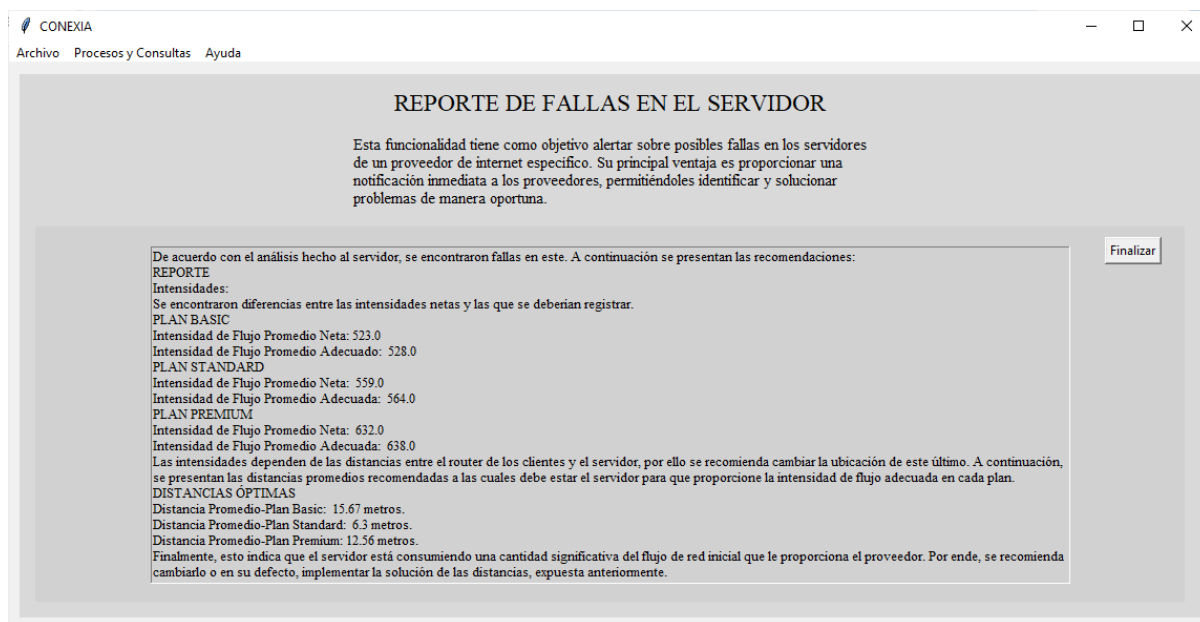
Distancia Promedio-Plan Standard: 7.12 metros.

Distancia Promedio-Plan Premium: 8.68 metros.

Finalmente, esto indica que el servidor está consumiendo una cantidad significativa del flujo de red inicial que le proporciona el proveedor. Por ende, se recomienda cambiarlo o en su defecto, implementar la solución de las distancias, expuesta anteriormente.

Finalizar

Para la opción 2 (sede B en Tigo):



CONEXIA

Archivo Procesos y Consultas Ayuda

REPORTE DE FALLAS EN EL SERVIDOR

Esta funcionalidad tiene como objetivo alertar sobre posibles fallas en los servidores de un proveedor de internet específico. Su principal ventaja es proporcionar una notificación inmediata a los proveedores, permitiéndoles identificar y solucionar problemas de manera oportuna.

De acuerdo con el análisis hecho al servidor, se encontraron fallas en este. A continuación se presentan las recomendaciones:

REPORTE
Intensidades:
 Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.

PLAN BASIC
 Intensidad de Flujo Promedio Neta: 523.0
 Intensidad de Flujo Promedio Adecuada: 528.0

PLAN STANDARD
 Intensidad de Flujo Promedio Neta: 559.0
 Intensidad de Flujo Promedio Adecuada: 564.0

PLAN PREMIUM
 Intensidad de Flujo Promedio Neta: 632.0
 Intensidad de Flujo Promedio Adecuada: 638.0

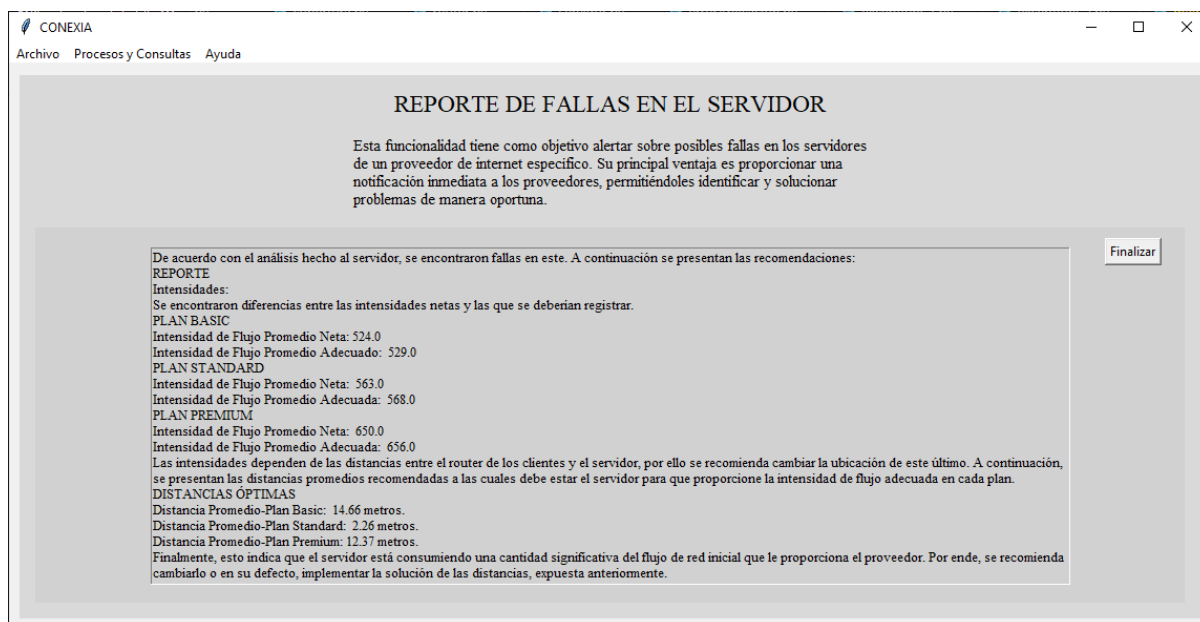
Las intensidades dependen de las distancias entre el router de los clientes y el servidor, por ello se recomienda cambiar la ubicación de este último. A continuación, se presentan las distancias promedio recomendadas a las cuales debe estar el servidor para que proporcione la intensidad de flujo adecuada en cada plan.

DISTANCIAS ÓPTIMAS
 Distancia Promedio-Plan Basic: 15.67 metros.
 Distancia Promedio-Plan Standard: 6.3 metros.
 Distancia Promedio-Plan Premium: 12.56 metros.

Finalmente, esto indica que el servidor está consumiendo una cantidad significativa del flujo de red inicial que le proporciona el proveedor. Por ende, se recomienda cambiarlo o en su defecto, implementar la solución de las distancias, expuesta anteriormente.

Finalizar

Para la opción 3 (sede C en Tigo):



CONEXIA

Archivo Procesos y Consultas Ayuda

REPORTE DE FALLAS EN EL SERVIDOR

Esta funcionalidad tiene como objetivo alertar sobre posibles fallas en los servidores de un proveedor de internet específico. Su principal ventaja es proporcionar una notificación inmediata a los proveedores, permitiéndoles identificar y solucionar problemas de manera oportuna.

De acuerdo con el análisis hecho al servidor, se encontraron fallas en este. A continuación se presentan las recomendaciones:

REPORTE
Intensidades:
 Se encontraron diferencias entre las intensidades netas y las que se deberían registrar.

PLAN BASIC
 Intensidad de Flujo Promedio Neta: 524.0
 Intensidad de Flujo Promedio Adecuada: 529.0

PLAN STANDARD
 Intensidad de Flujo Promedio Neta: 563.0
 Intensidad de Flujo Promedio Adecuada: 568.0

PLAN PREMIUM
 Intensidad de Flujo Promedio Neta: 650.0
 Intensidad de Flujo Promedio Adecuada: 656.0

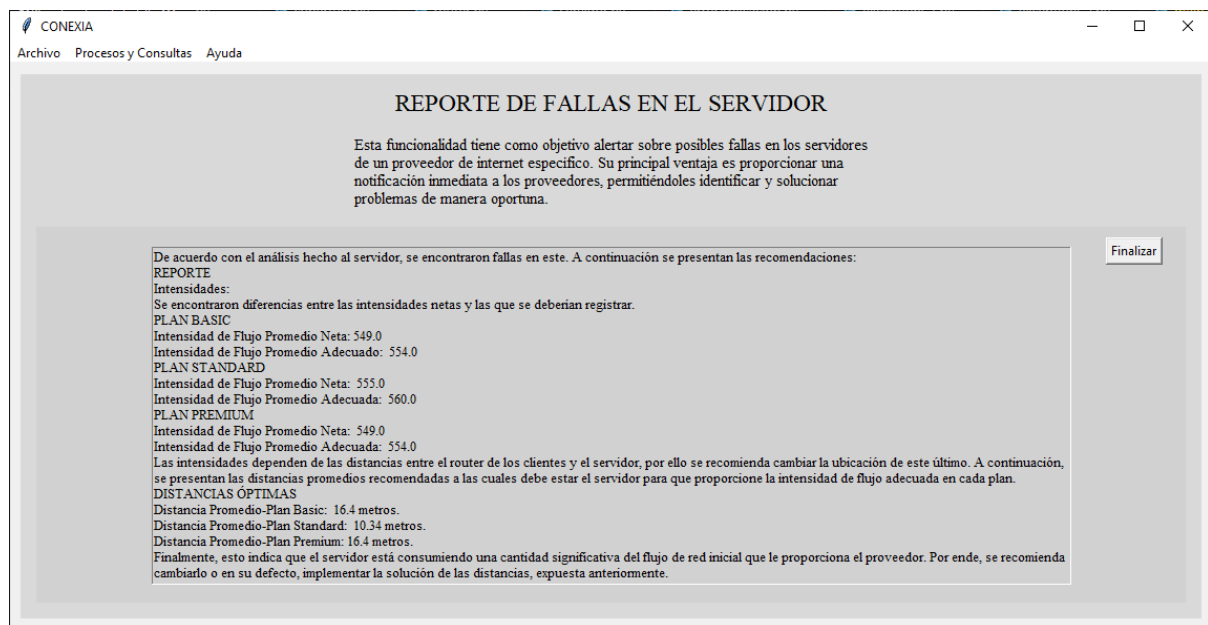
Las intensidades dependen de las distancias entre el router de los clientes y el servidor, por ello se recomienda cambiar la ubicación de este último. A continuación, se presentan las distancias promedio recomendadas a las cuales debe estar el servidor para que proporcione la intensidad de flujo adecuada en cada plan.

DISTANCIAS ÓPTIMAS
 Distancia Promedio-Plan Basic: 14.66 metros.
 Distancia Promedio-Plan Standard: 2.26 metros.
 Distancia Promedio-Plan Premium: 12.37 metros.

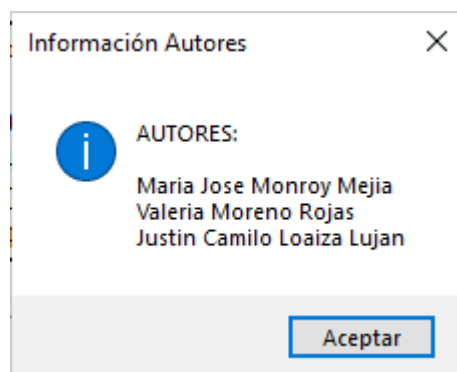
Finalmente, esto indica que el servidor está consumiendo una cantidad significativa del flujo de red inicial que le proporciona el proveedor. Por ende, se recomienda cambiarlo o en su defecto, implementar la solución de las distancias, expuesta anteriormente.

Finalizar

Para la opción 4 (sede D en Tigo):



En la opción 'ayuda' se desprende un menú con la opción 'Acerca de'. Al dar click en ella se verá que una ventana emergente aparece con la información de los autores de la aplicación.



En caso de que el usuario quiera verificar el correcto funcionamiento de la aplicación, puede seguir las siguientes recomendaciones en cada funcionalidad:

1. Funcionalidad Adquisición de un plan: puede crearse un usuario desde cero con datos aleatorios. No se necesita tener historial.
2. Funcionalidad Visualizar los dispositivos conectados: pueden usarse los datos de Id: 1234, sede: A, nombre: Andrea y proveedor: Tigo, en la consola.

Este es un buen ejemplo, los datos pertenecen a Andrea, un cliente registrado con una amplia variedad de dispositivos conectados y 7 pagos retrasados. Ella necesita un abono mínimo de 275.000 (para cumplir la condición de no tener +2 deudas) para visualizar los dispositivos, se ejecutaría el ítem.

3. Funcionalidad Mejora tu plan: el usuario del ejemplo anterior es un buen ejemplo aquí también, se ingresan los datos de Id: 1234, sede: A, nombre: Andrea y proveedor: Tigo, en la consola.

El plan sería mejorado de Tigo a Movistar, seguiría siendo de tipo Standard.

4. Funcionalidad Test: existen varios usuarios ya creados que se pueden utilizar aquí:
 - En caso de saturación puede usarse a nombre: Alejandra, proveedor: Movistar, sede: A, Id: 333.
 - En caso de incompatibilidad puede usarse a nombre: Gen, proveedor: Movistar, sede: C, Id: 2777.
 - En caso de inaccesibilidad puede usarse a nombre: Cob, proveedor: Tigo, sede: A, Id: 0.
 - En caso de que el test esté bueno: se solucionan los problemas de alguno de los usuarios anteriores (Alejandra, Gen o Cob) y se comprueba aquí se haya solucionado correctamente.
5. Funcionalidad Reporte de fallas en el servidor: el único dato de entrada que se necesita aquí es el nombre del proveedor y ya los conocemos: Tigo, Claro, Wom, Movistar. No es necesario tener historial ni algún dato adicional para ejecutar.